FEDERAL STATE AUTONOMOUS
EDUCATIONAL INSTITUTION OF HIGHER EDUCATION

NATIONAL RESEARCH UNIVERSITY
"HIGHER SCHOOL OF ECONOMICS"

Faculty of Computer Science

Marwan Bouabidi

BRAIN ACTIVITY TIME SERIES ANALYSIS WITH NEURAL NETWORKS AND INTERPRETABLE MACHINE LEARNING

MASTER'S THESIS

Field of Study "01.04.02 Applied Mathematics and Informatics"

Educational Programme "Artificial Intelligence and Data Analysis"

Supervisor:
Dmitry I. Ignatov

Moscow, 2025

Acknowledgments

I humbly dedicate this section to express my sincere gratitude to all those who have helped me and guided throughout writing this work. A special thanks to my family, my future spouse, and my friends.

Abstract

This study investigates different deep learning architectures for classifying brain states based on Electroencephalography (EEG) signals recorded during visual perception tasks involving varying brightness levels of two visual stimuli, the Necker cube and the MonaLisa. The raw EEG data was separated into five standard frequency bands (delta δ, theta θ, alpha α, beta β and gamma γ) with a 4th order Butterworth filter, each band representing a separate dataset for a given visual stimuli. Understanding this type of brain activity time series is crucial for both neuroscience research and some real-world applications like Brain-Computer Interfaces (BCIs), hence my motivation for this research. I evaluated a standard Artificial Neural Network (ANN), a novel hybrid Recurrent Neural Network (RNN) incorporating a convolutional front-end and attention mechanisms, and a CNN-Transformer model based on EEGNet (a convolutional neural network designed for general EEG data classification) and the Transformer model. The continuous EEG data was processed in short temporal sequences, containing 400 ms worth of data, making the classification task more challenging but also more relevant for real-world applications. I compared the models' performances using standard quantitative metrics (accuracy, F1, precision, recall and training time), and analysed their training and validation curves to get some insight into their learning dynamics. As for interpretability, to better understand why the models made their predictions, I employed the SHAP (SHapley Additive exPlanation) technique, through its Python library, to identify the most influential EEG channels across the studied frequency bands.

Results showed clear differences in model performance and generalisation ability across the five frequency bands and two visual stimuli. The standard ANN faced some severe overfitting problems, but still generally reached some moderate accuracies (for example, around 63%-64% in the beta band for both stimuli), although it presented the lowest performance overall. The new models, proposed in this study, to handle time sequences (the hybrid RNN and CNN-Transformer) performed significantly better. They achieved higher accuracies almost across all bands and stimuli, and improved generalisation by limiting overfitting. For example, the hybrid RNN reached accuracies exceeding 80% in the beta and gamma bands, while the CNN-Transformer was also highly competitive, surpassing the RNN in rare occasions under some specific conditions (cube-alpha and monalisa-beta). Both of these models also significantly improved training times (7-8 times faster), despite having comparable or fewer parameters. But the ANN still demonstrated some slightly superior performance over the advanced models in the monalisa-theta condition. The SHAP

analysis revealed that different models focused on distinct brain regions and activity patterns when classifying brightness levels, providing valuable and interpretable insights into their decision-making processes. These patterns could potentially reflect how the brain processes visual information under a variety of brightness conditions and visual stimuli complexity. This research highlights the importance of selecting appropriate deep learning architectures for EEG data and demonstrates the value of interpretable machine learning techniques in complex brain signal analysis.

Keywords: EEG, Visual Perception, Brightness Classification, Deep Learning, Artificial Neural Network, Recurrent Neural Network, CNN-Transformer, Explainable AI, SHAP, Brain-Computer Interface.

Contents

Acknowledgments i

Abstract ii

List of Figures vi

List of Tables viii

Nomenclature ix

1 Introduction 1

2 Methodology 5

2.1 Datasets . . . 6

2.1.1 Introducing the datasets . . . 6

2.1.2 Python processing . . . 10

2.2 Models . . . 15

2.2.1 Artificial Neural Network (ANN): base model . . . 15

2.2.2 Recurrent Neural Network (RNN): a hybrid model . . . 17

2.2.3 Convolutional Neural Network (CNN): CNN-Transformer model . . . 23

2.3 SHAP (Shapley Additive exPlanation) . . . 25

3 Results and analysis 26

3.1 Qualitative analysis of the ANN model performance . . . 26

3.2 Qualitative analysis of the hybrid RNN model performance . . . 30

3.3 Qualitative analysis of the CNN-Transformer model performance . . . 34

3.4 Analysis of confusion matrices of the top performing frequency bands . . . 38

3.5 SHAP Analysis . . . 41

3.5.1 Overall feature importance . . . 41

3.5.2 Channel-wise SHAP analysis for the lowest and highest brightness classes 43

4 Conclusion 49

5 Limitations and future work 51

Disclaimer 51

References **53**

A Appendix **57**

A.1 Data split for training 57

A.2 Analysis process 58

A.3 RNN-based model NAS results for alpha band of the cube dataset 59

A.4 Code 65

A.4.1 ANN 65

A.4.2 Hybrid RNN 66

A.4.3 CNN-Transformer 71

List of Figures

1 The experimental setup for the EEG measurements (Pisarchik et al., 2019). . . 7
2 Brain map of the 31 EEG channels used in the experiments (Islam et al., 2022). 7
3 Experimental protocols (Islam et al., 2022). 8
4 RNN-based model structure. 20
5 Delta band accuracy. 27
6 Theta band accuracy. 27
7 Alpha band accuracy. 27
8 Beta band accuracy. 28
9 Gamma band accuracy. 28
10 Delta band accuracy. 30
11 Theta band accuracy. 31
12 Alpha band accuracy. 31
13 Beta band accuracy. 31
14 Gamma band accuracy. 32
15 Delta band accuracy. 34
16 Theta band accuracy. 35
17 Alpha band accuracy. 35
18 Beta band accuracy. 35
19 Gamma band accuracy. 36
20 Confusion matrices for the ANN. 39
21 Confusion matrices for the hybrid RNN. 39
22 Confusion matrices for the CNN-Transformer. 39
23 Top 10 feature importance for the ANN. 41
24 Top 10 feature importance for the hybrid RNN. 42
25 Top 10 feature importance for the CNN-Transformer. 42
26 Brain map of the 31 EEG channels used in the experiments (Islam et al., 2022). 43
27 Channel-wise SHAP summary for ANN-Cube-Beta. 44
28 Channel-wise SHAP summary for RNN-Cube-Beta. 44
29 Channel-wise SHAP summary for CNN-Cube-Beta. 45
30 Channel-wise SHAP summary for ANN-Monalisa-Gamma. 47
31 Channel-wise SHAP summary for RNN-Monalisa-Gamma. 47
32 Channel-wise SHAP summary for CNN-Monalisa-Gamma. 48

33 GitHub repository. 65

List of Tables

1 Test results for the Cube dataset with the base ANN model 30
2 Test results for the Monalisa dataset with the base ANN model 30
3 Test results for the Cube dataset with the hybrid RNN model 34
4 Test results for the Monalisa dataset with the hybrid RNN model 34
5 Test results for the Cube dataset with the CNN-Transformer model 38
6 Test results for the Monalisa dataset with the CNN-Transformer model 38

Nomenclature

AI: Artificial Intelligence

ML: Machine Learning

DL: Deep Learning

XAI: Explainable Artificial Intelligence

BCI: Brain-Computer Interface

EEG: Electroencephalography

MEG: Magnetoencephalography

fMRI: functional Magnetic Resonance Imaging

ERD: Event-Related Desynchronisation

LRP: Layer-wise Relevance Propagation

ERP: Event-Related Potential

ICA: Independent Component Analysis

SHAP: Shapley Additive Explanations

NLP: Natural Language Processing

SVM: Support Vector Machine

ANN: Artificial Neural Network

RNN: Recurrent Neural Network

CNN: Convolutional Neural Network

LSTM: Long Short-Term Memory

GRU: Gated Recurrent Unit

DBN: Deep Belief Network

NAS: Neural Architecture Search

STFT: Short-Time Fourier Transform

GPU: Graphics Processing Unit

1 Introduction

Artificial Intelligence (AI) has experienced a historic surge in popularity in recent years, and has been evolving at a significantly high pace. This rise, allowed a diverse range of applications of its advanced tools to boost productivity in all kinds of sectors, and in healthcare in particular. Popular applications of AI in medicine include the analysis of medical imagery, such as X-ray scans or CT (computed tomography) scans, the reconstruction of unfinished medical imagery, blob detection related to cancer diagnosis, and many more (Bi et al., 2019). In the field of neuroscience as well, where the main focus is on the brain's structure and activity, AI has been making some significant changes. The visualisation of these specific activities, called neurovisualisation, can be achieved using a number of medical techniques; fMRI or functional Magnetic Resonance Imaging, MEG or Magnetoencephalography, and most importantly for this study: EEG or Electroencephalography. Gui et al. (2010) provide an excellent in-depth study that delves into more details about these techniques and their potential applications. The brain's electrical signals are its main way of sending information, and recordings of these signals are considered essential tools in neuroscience, as they are practically involved in the majority of research. For example, there is real-time neuroactivity monitoring involving patients under certain cognitive conditions that require constant attention; Also the use of these signals for disease prediction purposes (prevention). The work of Hussain and Park (2020) is an even more relevant practical example. Their study presents their affordable, EEG-based monitoring solution for stroke patients, known simply as HealthSOS, which forecasts stroke risk by detecting subtle neurological shifts and changes in brain wave activity.

Numerous studies have investigated the influence of behaviour and external stimuli on brain activity, focusing notably on EEG and MEG signals. These investigations reveal how such signals reflect the brain's responses to various types of cognitive and sensory demands. Thus, having the ability to conduct high-quality analysis of these signals is essential for comprehending the underlying behaviour behind it. Arutiunian et al. (2023), for instance, investigated the dynamics of sensory processing in children with autism spectrum disorder (ASD) using MEG signals. Since children with ASD often experience difficulties processing information around them, the study examined event-related desynchronisation (ERD) in the alpha band (8-12 Hz) during the simultaneous presentation of auditory and visual stimuli. The results showed a significant reduction in alpha ERD within the auditory cortex of children with ASD compared to their typically developing peers. This suggests reduced neural

engagement in auditory processing in the ASD group, unlike with their visual processing. Furthermore, unusual auditory ERD was linked to more severe communication problems, confirming its importance in understanding AD-specific sensory and social difficulties in children with ASD. Similarly, Yurchenko et al. (2023) explored the neural mechanisms behind register switching, defined as the process of shifting between formal and informal language, using EEG and event-related potentials (ERPs). The authors compared sentences containing register-incongruent words (such as slang in formal contexts) with sentences that were semantically incongruent (sentences grammatically correct but without sense). Their study revealed that register switching causes a pronounced N400 effect, indicating the brain's difficulty in integrating a word into an unsuitable context that does not fit with it. This leads to increased lexical-semantic processing demands in the brain, similar to what is seen with semantic anomalies (sentences without sense), but without the conflict resolution observed in figures of speech like metaphors or irony, where the brain manages to find a deeper meaning to what is seemingly senseless.

Fortunately, Machine Learning (ML) provides a wide range of advanced tools to deal with this kind of complex data. These techniques have significantly and rapidly evolved over the past few years, thus becoming more sophisticated and more performant than ever before. Currently, it has the capabilities to deal with the most complicated of tasks and provide reliable and useful results. In particular, ML allowed a newer and higher level of understanding of neurological signals in neuroscience, as it is able to capture complex and intricate patterns in data that the human eye mostly fail to notice. But performance alone is not sufficient when it comes to medical and healthcare-related tasks, unfortunately, as it is ethically unacceptable to let a "black box" type of system make decisions involving the health of living patients without understanding how. Physicians and medical experts often, and justifiably, require more transparency and more clarity from ML models in their approaches of problem-solving (Abdullah et al., 2021); In other words, they need models that offer help in the decision-making process by providing understandable and interpretable decisions. These models are also known as "explainable" ML models, part of the larger concept of explainable artificial intelligence or XAI. There are several distinct and informative definitions of explainable ML models available in literature, but they all agree on one fundamental, unchangeable point: such models must allow users to understand the reasoning behind their predictions to a certain acceptable degree (Roscher et al., 2020; Belle and Papantonis, 2021).

Arguably, the most popular real-world applications of brain activity signals are Brain-Computer Interfaces (BCIs). These systems bypass the need for physical movement by using

cognitive signals to facilitate direct communication or control of external devices (McFarland and Wolpaw, 2017). BCIs encompass a wide range of applications, from controlling robotic parts or limbs to communicating with brain microchips; today, they are increasingly becoming closer to reality than to science fiction. The well-known Neuralink chip, for instance, is a great example of a BCI being developed for public everyday use (Musk et al., 2019). Among them, EEG-based BCIs are the most common, primarily due to their non-invasive nature and their real-time operation capacity. EEG captures electrical brain signals via electrodes placed on the scalp; and then these signals undergo preprocessing, filtering, and appropriate transformations, before being passed to ML models which aim to perform specific, context-bound tasks, such as classification or regression.

A new question thus arises: what ML model can be considered as "the most approprite" for a classification task involving brain waves (EEG in particular)? For complex tasks dealing with high-dimensional data, such as EEG signals, Convolutional Neural Networks (CNNs), initially introduced by LeCun et al. (1998), are a popular and reliable choice. These networks have demonstrated significant capabilities, especially when it comes to image analysis and Natural Language Processing (NLP). However, this does not always translate directly to electrical brain signals, where achieving high accuracy can bring a high computational cost. Therefore, while effective, CNN models can be overly complex and computationally-demanding to meet the desired quality of performance. For example, Zhou et al. (2018) investigated in their research the application of CNNs to raw EEG signals for the detection of epileptic seizures. They used two datasets for that: one containing intracranial EEGs from the University Hospital of Freiburg, Germany; the other is an open-source database (CHB-MIT, USA) collected from 23 subjects. Despite using a relatively simple CNN architecture with only three hidden layers, their results were notably impressive: average accuracy rates exceeded 90% across all experiments and both datasets when using frequency domain signals for classifying interictal, preictal, and ictal segments. With time domain signals, performance was slightly lower, with accuracies ranging from 83.8% to 91.1% for the Freiburg dataset, and from 47.9% to 59.5% for the CHB-MIT dataset. These results highlights the potential of CNNs even with limited data. Additionally, Craik et al. (2019) conducted a comprehensive review of 90 studies on deep learning techniques in EEG data classification, covering various tasks: emotion recognition, assessment of mental workload, detecting seizures, mental imagery and sleep scoring. The authors concluded the following: while specific DL models might excel and slightly outshine others in specific task(s), CNNs, alongside Deep Belief Networks (DBNs) generally presented the best performance overall across all the tasks examined in the

study. Xu et al. (2019) explored this topic also and provided a relevant example that offers a different perspective and new insights concerning the limitations of CNNs. The authors investigated the classification of EEG signals related to motor imagery (MI), and they did so using a model that combines a pre-trained CNN (VGG-16) and a target CNN which received the parameters of the pre-trained VGG-16, in order to reduce an often-long and demanding training phase. EEG signals were transformed, with the help of Short-Time Fourier Transform (STFT), into time-frequency spectrum images, and then were passed to the pre-trained neural network for further training. This custom model unsurprisingly outperformed other ML models such as Support Vector Machine (SVM), Artificial Neural Network (ANN), and even an untrained CNN. But one of the main limitations encountered in this research was still the time-consuming task of training, even while using a powerful GPU (Graphics Processing Unit) and a pre-trained CNN that helped them avoid training from zero.

Islam et al. (2022) focused as well on the application of ML methods to classify brain states in their study, particularly relying on EEG signals obtained through visual perception. The aim being a balance between performance and interpretability, since interpretations are critical for neuroscience applications (and for all healthcare generally), as has been discussed previously. The authors of this work used two datasets containing various EEG recordings from 31 channels with 250 Hz sampling rate (250 temporal points recorded per second); To get these recordings, brain activity was captured while participants were exposed to two distinct visual stimuli, the **Necker cube** and the Mona Lisa portrait, at 10 distinct levels of brightness separately presented to them for a minute each. The collected raw EEG signals were then filtered into 5 standard frequency bands (delta, theta, alpha, beta, and gamma). Adequate preprocessing was also applied to ensure reliable signal quality. The study used ANN based models with a fixed architecture (layers) but with a wide range of customisation involving activation functions and optimisers, all while maintaining interpretability through the use of SHAP (Shapley Additive Explanations) framework. This Python library is designed for explaining machine learning models by assigning scores to data features using Shapley values from cooperative game theory. It gives an insight on the contribution of different features (or data characteristics) to the actual prediction Lundberg and Lee (2017); Rozemberczki et al. (2022). Using this library, the authors were able to identify the most influential features, specific hemispheric channels from the 31 channels used, in the classification process. They also compared 8 optimisation algorithms, including Adagrad, Adam, and Nadam, and found that Adagrad presented the most consistent performance across the different frequency bands and activation function choices. The results showed that Adagrad optimiser achieved

best results across different frequency bands and activation functions. But despite the high classification accuracy achieved, the study pointed out to the challenge of overfitting faced by some of the models.

The aforementioned works on brain activity classification with ML have effectively highlighted areas for potential further investigation. One potential avenue for further research could involve examining alternative deep learning architectures, such as transformer-based or attention-based models, to enhance performance and efficiency; Exploring other machine learning approaches is also another option. This leads to the current study, focusing on visual perception-based EEG signals, and whose purpose is to build on the foundational study of Islam et al. (2022) by researching brain signal classification with different DL models. In this specific context, my main questions are: **what processing steps should the data go through in this study? Which Deep Learning models could be suitable for this classification task? And how can the classification results be interpreted?**
In this work, to answer these research questions, I will analyse and classify the 2 EEG datasets from Islam et al. (2022): the "Mona Lisa"-related EEG signals, and the "Necker cube"-related EEG signals. I will attempt to incorporate improvements through developing novel DL architecture(s) and comparing the outcome to the relevant existing architecture(s) from literature, quantitatively and qualitatively; all while retaining interpretability through the use of SHAP library.

2 Methodology

EEG signals are electrical activities produced by the activation of a large number of neurons in the brain, in the cerebral cortex region specifically. These brain signals are known for their high temporal resolution, measured in milliseconds usually, which makes them suitable for real-time applications, in particular for the aforementioned Brain-Computer Interfaces or BCIs.

EEG-based BCIs rely on two key steps: **feature extraction** and **feature translation**. Feature extraction on one hand identifies relevant patterns in EEG signals, such as amplitude, frequency, or phase information, often enhanced by spatial filtering methods like the surface Laplacian or common spatial patterns (CSP). Newer and more sophisticated approaches like convolutional neural networks, are more and more used recently in order to efficiently extract features from raw EEG datasets. Feature translation on the other hand maps these features to specific commands using ML algorithms, for example support vector machines

(SVMs) or deep learning models, to classify EEG patterns into different user intentions. A major challenge in EEG-based BCIs is related to the non-static nature of brain waves; they can be affected by a number of factors such as fatigue, attention, or changes in mental state. Therefore, evolving algorithms capable of continuously updating their parameters are required to maintain quality and performance over time (McFarland and Wolpaw, 2017).

The current work will cover both steps, feature extraction by preprocessing and identifying relevant EEG features, and feature translation using ML models to classify brain states based on those features. The focus is on creating interpretable models that can classify these states related to visual perception, while also maintaining an understanding of how different features contribute to the classification process. Let us firstly introduce the EEG datasets of interest to this research:

2.1 Datasets

2.1.1 Introducing the datasets

The datasets used in this paper were produced by Islam et al. (2022) to investigate brain activity during visual perception tasks using EEG data. Signals were recorded across 31 channels at a 250 Hz sampling rate with the BE Plus LTM amplifier (made by EB Neuro S.p.A., Florence, Italy). The experiments focused on understanding how the brain processes visual stimuli of varying brightness levels, aiming to classify brain states associated with these tasks. The experiments were fully realised in the aforementioned study, and this current work does not replicate those experiments but rather only focuses on the study of the obtained results using ML tools, in particular the task of classifying EEG signals.

The participants were exposed to two types of visual stimuli: **the Necker cube**, a geometric figure known for its perceptual ambiguity, and **the Mona Lisa**, the classic artwork of Leonardo da Vinci. Each stimulus was shown at 10 brightness levels, ranging from 0.1 to 1.0, increasing in equal intervals. For each brightness level, participants observed the image for 60 seconds, followed by a 20-second rest period. Data was collected from five participants, with each individual completing experiments for all brightness levels and for both stimulus types. Hence, two datasets, each related to a visual stimuli, were produced. The preprocessing phase performed by the authors, Islam et al. (2022), included removing artifacts like eye blinks and muscle movements using Independent Component Analysis or ICA (Makeig et al., 1995), as well as filtering with a fourth-order Butterworth bandpass filter (100 Hz) and a 50 Hz notch filter to reduce noise. The experimental design was carefully

done, with respect to ethical guidelines outlined in the Helsinki Declaration and approved by the Ethics Committee of Kant Baltic Federal University, and with consent provided by the participants before taking part in the study.

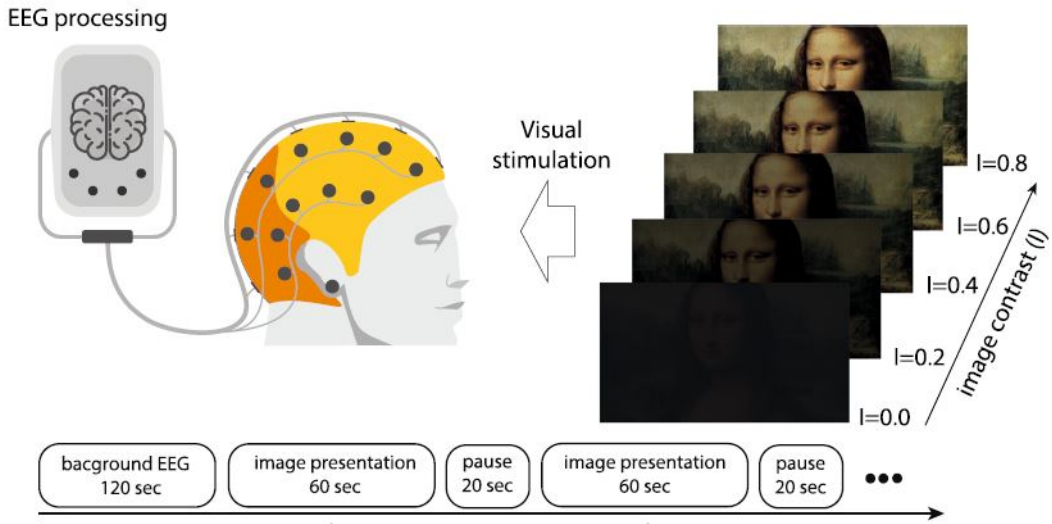


Figure 1: The experimental setup for the EEG measurements (Pisarchik et al., 2019).

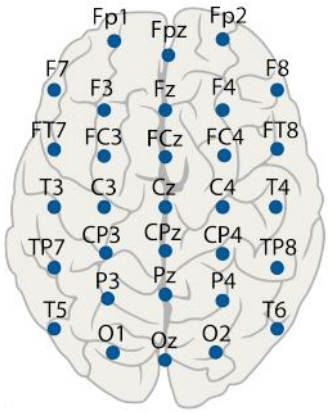


Figure 2: Brain map of the 31 EEG channels used in the experiments (Islam et al., 2022).

The raw EEG data, electrical potential readings in micro-volts (μV), was recorded across 31 channels with a sampling rate of 250 Hz (250 samples per second), and the duration of a single EEG measurement or recording is approximately 60 seconds. This means that for one measurement (one participant, one visual stimuli and one brightness level) there will be around $250 * 60 = 15000$ time points or **brain states**, and the data should be of shape $\approx (15000, 31)$. In the available data, there is usually more than 15000 time points since the duration of the recording is not always exactly 60 seconds, but could exceed it with a few hundred milliseconds due to human error, hence the difference. The **target variable** in this data is the **brightness level I**, which takes 10 possible values from 0.1 to 1.0 by a 0.1 increment,

and this means 10 possible classes for each data point. The classes are enumerated from 0 to 9, respectively corresponding to the brightness levels in increasing order. In addition, in a full experiment with a selected participant and a selected visual stimuli, EEG background activities have been measured before and after it for a duration of 120 seconds (Islam et al., 2022; Pisarchik et al., 2019).

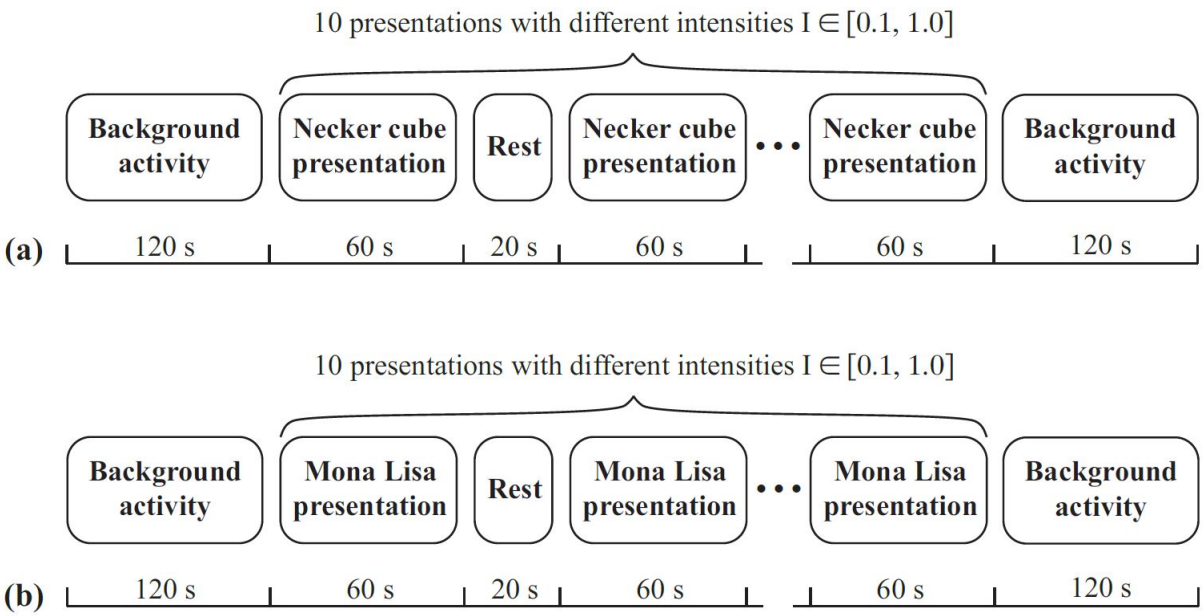


Figure 3: Experimental protocols (Islam et al., 2022).

The variety of the visual stimuli in the datasets is particularly interesting; the Necker cube challenges the brain with perceptual ambiguity, while the Mona Lisa evokes visual and artistic appreciation. The gradual increase in brightness allows the observer to gradually distinguish more details as the experiment goes on, by going through different levels of perception; from a low or basic level while trying to recognise the image itself, to a higher level trying to notice the smaller details in it. Furthermore, the high temporal resolution of the EEG recordings provides a diverse range of brain states for ML analysis in order to identify the most relevant EEG features.

EEG signals contain a whole spectrum of neural activity, but not all frequency components are relevant for a cognitive task. To focus on meaningful patterns the EEG signals were processed into five standard frequency bands, which is a standard approach for feature extraction (Mosher et al., 1999). Each band reflects different aspects of brain activity, such as relaxation, focus, or cognitive engagement (Kumar and Bhuvaneswari, 2012). Here below is a brief description of the range and significance of each band:

- delta δ (1-4 Hz): associated with deep sleep and restorative processes; often observed during slow-wave sleep. It is prominent in infants and during healing or recovery, but

abnormal for awake adults.

- theta θ (5-8 Hz): linked to light sleep, relaxation, and meditative states. It also plays a role in creativity, emotional processing, and the encoding of memories, particularly during drowsy or transitional states. It is abnormal for adults, but normal for children under 13.
- alpha α (9-12 Hz): represents relaxed wakefulness, calmness, and closed-eye relaxation. Alpha waves are often dominant when individuals are calm yet alert, and they act as a bridge between the conscious and the subconscious mind.
- beta β (13-30 Hz): associated with conscious behaviour and action, and related to what the senses receive. Appears for example in active thinking, problem-solving, decision-making, and focus. An elevated beta wave activity can indicate states of heightened alertness but may also be linked with stress or anxiety if strongly dominant.
- gamma γ (31-45 Hz): connected to high-level cognitive functioning by combining senses and memory. This includes learning, memory consolidation, and information processing. Gamma waves are often associated with moments of insight, heightened perception, and the integration of information across brain regions.

The raw signals , $\mathbf{X} \in \mathbb{R}^{n_temporal_points \times n_channels}$, were first preprocessed by reassigning them to the total average and subtracting the mean, removing any noises.

- **Referencing (spatial normalisation)**: Raw EEG signals (V_{raw}) were re-referenced using a common average approach:

$$V_{ref} = V_{raw} - \frac{1}{N} \sum_{j=1}^{N} V_{raw,\, j} \tag{1}$$

  Where $N = n_channels$, the number of electrodes or channels. This eliminates system noise common to all channels while preserving neural signals.

- **Centering (temporal normalisation)**: applied to remove channel's DC offset. They are baseline shifts in the recorded voltage due to a number of non-neurological factors, such as Electrode-Skin interface effects, amplifier shifts, or even environmental factors during the experiment. This is done through temporal mean subtraction (Luck, 2014):

$$V_{centered} = V_{ref} - \mathbb{E}_t[V_{ref}] \tag{2}$$

This ensures signals with a zero mean for frequency analysis and filter use.

Each frequency band of interest was then isolated using a **fourth-order Butterworth bandpass** filter, with the lower and upper cut-off frequencies (f_{Low} and f_{High}) chosen according to a defined frequency band. This step ensures that only the components within the desired frequency range are retained, suppressing irrelevant frequencies. Filtering is essential because each frequency band is associated with different brain functions, as we have seen previously, and retaining the full EEG spectrum may obscure the role each band plays in the neural activity. By separating the signals, it will be possible to analyse each band's contributions to the classification of brain states separately; The filtered data will then be fed to the ML models, and this improves classification accuracy, as the models can focus on the most informative components rather than on processing redundant or misleading features from the full signal. This also improves the interpretability of the classification process. The data was collected in compliance with the Helsinki Declaration, as stated previously, and is publicly available for use and further exploration (Robiul et al., 2022).

2.1.2 Python processing

A major change that I incorporated compared to the reference study of Islam et al. (2022) is in the data processing phase before using it with the DL models. The approach adopted in their study, while giving impressively high classification accuracies quickly reaching a perfect score of 1, presents a few limitations that hinders its generalizability and practical applicability. In their setup (the code for this part can be found here), the EEG data from a single participant across 10 visual stimulus intensities (ranging from 0.1 to 1) are processed by stacking the full one-minute-long recordings, of approximately 15000 time points, for each channel and intensity. So, each data line is a vector of $15k$ points corresponding to one intensity class and one channel. These high-dimensional vectors are then used as input to the artificial neural network, which rapidly achieves perfect accuracy in classifying the corresponding stimulus intensity.

While there is no direct data leakage in this pipeline, the near-perfect performance can be attributed to the redundancy and consistency inherent in the data. These long 1-minute sequences hold a large amount of information and can act almost like unique fingerprints for each intensity. This means the model is able to quickly learn and match the overall patterns without needing to generalise across shorter or more variable data. In this approach, each long recording is treated as if it shows a consistent and stable response to the stimulus. In

other words, the model has only the task of trying to find patterns in a long input sequence that will lead it to the intensity level (a level out of 10) without worrying too much about the highly dynamic nature of EEG signals and their change over time. As a result, the classification task becomes much easier, which helps explain why the model reaches perfect accuracy so quickly. So, it seems that in this case the model is not efficiently learning the EEG data, as the network does not need to learn abstract or generalisable features; instead, it merely memorises distinct waveform patterns that are repeatedly presented across the different channels. Moreover, the use of a deep, over-parameterised architecture on such a low-complexity task, where only 10 discrete output classes exist, enables the model to fit the data almost perfectly after only a few training epochs.

After exploring this setup, it can be seen that it has some important limitations, especially when considering real-world applications such as brain-computer interfaces (BCIs). In practice, decisions in BCI systems need to be made quickly and based on short time windows. Often, only a few hundred milliseconds of data are available. My approach addresses this by using smaller time segments, such as 100 time points (0.4 seconds worth of data) across all EEG channels. This makes the task much more challenging for the model but also much more realistic and adaptable to real-time scenarios. It forces the model to learn more discriminative and robust features, rather than relying on long-term repetition or memorisation. And this will make it more reliable and applicable in dynamic and time-sensitive environments.

Now I will go through my data processing approach step by step. The original EEG data collected has been referenced, centred then filtered with **fourth-order Butterworth bandpass** filter, as indicated previously, using the following code:

```
from scipy.signal import butter, filtfilt, sosfiltfilt


def butter_bandpass_filter(data, lowcut, highcut, fs, order=2):
    sos = butter(N=order,
                 Wn=[lowcut, highcut],
                 btype='bandpass',
                 fs=fs,
                 output='sos')
    return sosfiltfilt(sos, data)


def filter_eeg_into_bands(df, fs, order=4):
```

```
# Define frequency bands
bands = {
    'delta': (1, 4),
    'theta': (5, 8),
    'alpha': (8, 12),
    'beta': (13, 30),
    'gamma': (31, 45)
}

# Initialize dictionary to store filtered data
filtered_dfs = {}

#Referencing
average_signal = df.mean(axis=1)
df_referenced = df.sub(average_signal, axis=0)

#Centering
df_centered = df_referenced - df_referenced.mean(axis=0)

# Apply bandpass filtering for each frequency band
for band, (lowcut, highcut) in bands.items():
    filtered_data = butter_bandpass_filter(df_centered.values,
                    lowcut, highcut, fs)
    filtered_df = pd.DataFrame(filtered_data, columns=df_centered.columns)
    filtered_dfs[band] = filtered_df
filtered_dfs['raw'] = df
filtered_dfs['full_centered'] = df_centered
return filtered_dfs
```

In the code above, **second-order sections (sos)** were used with $N = 2$, where N represents in this case the number of biquad sections of the desired system. And therefore, the resulting order will be $2 * N = 4$. The sos format decomposes the filter into cascaded second-order stages, which minimises numerical precision errors and improves stability. In addition, it is recommended by the developers here to use sos-filtering for general purpose

filtering. The data has then been merged for the five participants into two torch tensors, one for each dataset. Then, for each of the datasets the bandpass Butterworth filter was applied and resulted in 5 new tensors, each representing one of the frequency bands (δ, θ, α, β, γ). The tensors were then divided into 3 parts: 80% training, 10% validation, and 10% testing.

The order of the data points highly matters in the split. EEG data is a spatial (across channels) and temporal (across time periods) data, thus the splitting must preserve this temporal order while also making sure that the data is properly shuffled. In addition, for a fixed dataset and a fixed band, there are 5 participants each with 10 experiments (10 brightness levels), which gives us $5 * 10 = 50$ trials in total. It would not be the best idea to train the model, for example, on the first 40 trials, representing 4 of the participants, while validating on the 5 first brightness classes of the fifth participant, and testing on his 5 remaining brightness classes, as it would create imbalanced data on the participant-level, and also the validation/testing steps will not truly reflect the performance of the model. To address this issue, I proceeded as follows; First, the data for each dataset has been stacked trial by trial (each trial containing ≈ 15000 temporal data points), going from the 10 trials of the first participant, to the 10 trials of the last one. The entire dataset is then segmented into batches or temporal sequences (each of size 100, a fixed size empirically chosen) while preserving the internal temporal order. The trials do not overlap, since they have been recorded separately and are not directly related, that is why their data should not mix in one sequence. Thus, for every trial, the last remaining sequence is ignored before going to the next trial's data, since a random shuffle of sequences (or batches) of a fixed size with a few that are not of the same size will completely mess the temporal order I'm trying to preserve. Also, every trial is supposed to be, theoretically according to the original experimental setup, of a duration of exactly 1 minute, which will give exactly 15000 points using a recording frequency of 250 HZ, a number divisible by the chosen sequence size of a 100, therefore dropping the remaining points should not affect the results in any significant manner since they exist unintentionally due to human error. These sequences are then randomly shuffled, making sure that the train/validation/test splits are more balanced and representative of the entire data. Finally, the shuffled data is split into the desired proportions and converted into PyTorch DataLoaders for training. Here below is the Python code I used to achieve this shuffle:

```
def shuffle_batches(X, y, batch_size, trials = trial_sizes[dataset]):
    batches_X = []
```

```
    batches_y = []
    trial_boundaries = [0]
    #cumulative sizes of successive trials
    for size in trials:
        trial_boundaries.append(trial_boundaries[-1] + size)

    # Process each trial
    for i in range(len(trial_boundaries) - 1):
        start = trial_boundaries[i]
        end = trial_boundaries[i+1]
        trial_X = X[start:end]
        trial_y = y[start:end]
        n = trial_X.shape[0]
        num_full_batches = n // batch_size
        remainder = n % batch_size

        # Create full batches from this trial
        for j in range(num_full_batches):
            batch_X = trial_X[j * batch_size : (j+1) * batch_size]
            batch_y = trial_y[j * batch_size : (j+1) * batch_size]
            batches_X.append(batch_X)
            batches_y.append(batch_y)

    # Shuffle the list of batches
    indices = np.arange(len(batches_X))
    np.random.shuffle(indices)
    X_shuffled = np.concatenate([batches_X[i] for i in indices], axis=0)
    y_shuffled = np.concatenate([batches_y[i] for i in indices], axis=0)

    return X_shuffled, y_shuffled
```

A flow chart explaining the split process is provided in the appendix section A for further explanations, as well as another flow chart showing the whole analysis process in simplified steps.

2.2 Models

The first model to be tested is the Artificial Neural Network (ANN) suggested by Islam et al. (2022), a standard but large neural network with 8 hidden layers. The main aim here will be to first evaluate this model with the available data using a number of reliable metrics (such as accuracy, F1-score, execution time...), and then to design a new model and try to improve the previous model's results. Since only an ANN was explored in the aforementioned article, I decided to test Recurrent Neural Networks (LSTMs specifically) since they excel at dealing with time series data presenting high temporal correlation (Karim et al., 2017), as well as incorporating attention mechanisms to enhance it, and possibly also include a CNN (Convolutional Neural Network) front-end layer to form an improved **hybrid model**. I will also test a reliable CNN architecture designed for EEG data, **EEGNet**, enhanced with a transformer block, to see how it compares to both the ANN and the RNN.

The models were fully trained and tested on Google Colab's free **T4 GPU** cloud computer, which has the following hardware:

- System RAM: 12.7 GB
- GPU RAM: 15 GB
- Disk: 112.6 GB

The training process for each model has been repeated several times (re-trained from zero) to ensure the consistency of the obtained results.

2.2.1 Artificial Neural Network (ANN): base model

Artificial Neural Networks (ANNs) are computational models which draw their inspiration from biological neural system architectures. These systems detect advanced relationships by adapting their learning approach multiple times. ANNs consist of multiple interlinked nodes known as neurons which are structured into three separate layers including an input layer and at least one hidden layer and an output layer. Learning occurs through weight adjustments between neurons which let neural networks produce an approximate representations of non-linear patterns within the data. The learning process, which is achieved by back-propagation and similar algorithms, enables ANNs to improve their internal representations while they learn from provided examples. ANNs extract their knowledge directly from data instead of following traditional rule-based programming which suits complex tasks that do not exist with defined rules or have complicated rule systems to describe. This data-driven

approach provides useful progress in recognition systems as well as natural language processing and predictive analytics, this is because it allows the model to understand complex multidimensional relations and associations. The effectiveness of ANNs remains clear but researchers actively explore the specific processes by which models develop generalised wider skills from provided training information. It can be said therefore that ANNs serve as an effective computational framework for handling complex issues, while at the same time creating practical research opportunities for studying learning and representation phenomena (Goodfellow et al., 2016).

In this study, I will focus on the full combined data of all the participants for each of the two visual stimuli, which will give me two datasets of mixed EEG data. And each dataset will be decomposed into the 5 mentioned frequency bands with the Butterworth bandpass filter, as previously explained. This choice introduces variety to the data and will help me find a more general model architecture that is less exposed to overfitting issues and also less participant-related.

According to the findings of Islam et al. (2022), the best ANN architecture to deal with this EEG data is constituted of eight fully-connected dense hidden layers with a decreasing number of neurons in-between layers (2500, 1000, 500, 200, 100, 50, 25, and 15), followed by the final output layer consisting of only 10 neurons corresponding to the 10 classes representing the different intensity levels of the visual stimuli. The hidden layers are separated by the hyperbolic tangent (tanh) activation function to capture non-linearities in the data, and the output layer gives the final probability distribution over the 10 intensity classes. The input is a two-dimensional EEG-data tensor of shape $(batch_size, n_channels)$, combining batch_size ordered time points (as explained in the section 2.1.2) and all spatial channels into one batch, which allows the model to learn spatial patterns across the multiple channels in the data. In this case, the batch size for the ANN is the length of the temporal sequence itself since it only accepts two-dimensional data, unlike for the RNN or the CNN which expects higher-dimensional data. The limitation of this model mainly shows in the fact that it treats every data point in the batch, time point in this case, independently of the others, causing it to potentially miss some crucial temporal patterns (the approach of the reference paper of Islam et al. (2022), although takes care of the temporal dependency by taking all the time points in a given trial, treats every spatial channel independently, thus missing important spatial patterns), and this is unfortunately due to its core design. Despite this constraint, the ANN model is able to capture, to a certain degree, some interesting relationships across the datasets of interest to this work. Empirically, in the context of this study, the ANN performed

best receiving data of shape $(batch_size, n_channels)$.

After a number of experiments, the training options were fixed as follows:

- Batch size: 100. It represents 0.4 seconds worth of EEG data (since the sampling rate = 250 Hz).
- Optimiser: Adagrad.
- Learning rate: 5e-3.
- Number of epochs: 50.

These options provided stability and improved the model's training over the mixed EEG data of the 5 subjects. This model has $3,208,675$ trainable parameters.

2.2.2 Recurrent Neural Network (RNN): a hybrid model

Recurrent Neural Networks (RNNs) adopt traditional Artificial Neural Networks (ANNs) by adding memory elements that allow processing of sequential data. Standard feedforward ANNs do not include feedback loops yet RNNs incorporate them because of their ability to store data across sequential time steps. The temporal capability is possible through recurrent connections that transmit neuron outputs from time t to serve as inputs at time $t + 1$ for both self and other neurons. By having a recurrent structure, RNNs excel at recognizing dependencies inside sequences because this pattern further makes them capable of working with three main applications: time-series data and natural language processing together with speech recognition. RNNs feature an internal memory system that stores past input data thus affecting the processing approach for upcoming inputs. However, RNNs suffer from two major training difficulties because of their recurrent structure: this structure can lead to both gradient explosion and vanishing phenomena, as thoroughly explained by Pascanu et al. (2013). Researchers therefore developed LSTM and GRU networks as variants to solve these issues (Hochreiter and Schmidhuber, 1997; Cho et al., 2014). New network designs feature control systems that manage information movement and this enables better detection of extended relationships within data structures. LSTM and GRU networks, together with their improved regulation mechanisms, have brought substantial progress to sequence modeling because these architectures effectively implement temporal processing within neural networks. We can conclude that RNNs provide an effective way to process sequential information from time-varying data streams (or time series) through which meaningful patterns emerge. This capability was further demonstrated by Graves (2013) in their study on cursive-writing generation using RNNs. But the process of training these networks presents specific

difficulties. Continuous and sustained investigations therefore aim to redesign the network structure and learning procedures in order to expand its applications to wider domains.

The RNN-based model constructed in this study is a hybrid deep learning architecture that processes our multichannel EEG signals by passing it through different blocks, the core being the recurrent layers. The input tensor has the shape $\mathbf{X} \in \mathbb{R}^{B \times T \times C}$, where B is the batch size, T is the sequence length, and C is the number of channels. I start the processing with a convolutional front-end that operates on our input tensor after permuting his last two dimensions, transforming it from $\mathbb{R}^{B \times T \times C}$ to $\mathbb{R}^{B \times C \times T}$, by applying a one-dimensional convolution, followed by a one-dimensional batch normalisation. The CNN layer gives $H_{\text{conv1}} = 128$ features. A tanh activation function is applied to the normalized output, followed by a dropout layer with a dropout probability of 0.2, and the tensor is then permuted back to the shape $\mathbb{R}^{B \times T \times H_{\text{conv1}}}$.

The next stage represents the core of the model, and it involves three LSTM layers which are used to extract temporal dependencies from the signal. Each LSTM layer is configured to operate in a bidirectional manner to better understand the temporal sequences. The process can be summarised with the following set of equations:

$$
\begin{aligned}
\mathbf{H}^{(1)} &= \text{LSTM}^{(1)}(\mathbf{X}_{\text{conv}}), \quad \mathbf{H}^{(1)} \in \mathbb{R}^{B \times T \times H_1 \cdot d}, \\
\tilde{\mathbf{H}}^{(1)} &= \text{Dropout}\Big(\tanh\big(\text{LayerNorm}(\mathbf{H}^{(1)})\big)\Big), \quad \tilde{\mathbf{H}}^{(1)} \in \mathbb{R}^{B \times T \times H_1 \cdot d}, \\
\mathbf{H}^{(2)} &= \text{LSTM}^{(2)}(\tilde{\mathbf{H}}^{(1)}), \quad \mathbf{H}^{(2)} \in \mathbb{R}^{B \times T \times H_2 \cdot d}, \\
\tilde{\mathbf{H}}^{(2)} &= \text{Dropout}\Big(\tanh\big(\text{LayerNorm}(\mathbf{H}^{(2)})\big)\Big), \quad \tilde{\mathbf{H}}^{(2)} \in \mathbb{R}^{B \times T \times H_2 \cdot d}, \\
\mathbf{H}^{(3)} &= \text{LSTM}^{(3)}(\tilde{\mathbf{H}}^{(2)}), \quad \mathbf{H}^{(3)} \in \mathbb{R}^{B \times T \times H_3 \cdot d}, \\
\tilde{\mathbf{H}}^{(3)} &= \text{Dropout}\Big(\tanh\big(\text{LayerNorm}(\mathbf{H}^{(3)})\big)\Big), \quad \tilde{\mathbf{H}}^{(3)} \in \mathbb{R}^{B \times T \times H_3 \cdot d}.
\end{aligned}
\tag{3}
$$

Where $(H_1, H_2, H_3) = (128, 256, 128)$ denote the hidden sizes of the three LSTM layers, and $d = 2$ corresponds to the number of directions, as I used bidirectional LSTMs.

After the recurrent layers, an attention mechanism is applied to capture the most important temporal features. A multi-head scaled dot-product self-attention module, faithful to the original design from the widely known paper by Vaswani et al. (2017), is used, thus

transforming the output as follows:

$$
\begin{aligned}
&\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V, \\
&\mathbf{Q}_i = \mathbf{Q}\mathbf{P}_i^Q, \quad \mathbf{K}_i = \mathbf{K}\mathbf{P}_i^K, \quad \mathbf{V}_i = \mathbf{V}\mathbf{P}_i^V, \quad \text{for } i = 1, \ldots, h, \\
&\text{head}_i = \text{Dropout}\left(\text{softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{d_k}}\right)\right)\mathbf{V}_i, \\
&\text{MultiHead}(\mathbf{X}) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_h\right)\mathbf{W}^O.
\end{aligned}
\tag{4}
$$

Here, $\mathbf{X} \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ is the input sequence of hidden states, where B is the batch size, T is the sequence length, and d_{model} is the number of features (after passing through the recurrent layers). The number of attention heads is $h = 16$, and the dimension of each head is $d_k = \frac{d_{\text{model}}}{h}$. The projection matrices $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ map the input into query, key, and value spaces. The parameters of each head, $\mathbf{P}_i^Q, \mathbf{P}_i^K, \mathbf{P}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$, are used to split the projected features between the parallel attention heads h. The output of each head is concatenated and linearly transformed with $\mathbf{W}^O \in \mathbb{R}^{(h \cdot d_k) \times d_{\text{model}}}$ to restore the original dimensionality d_{model}. This attention transformation is followed by a mean pooling operation applied over the time dimension to obtain a fixed-length representation:

$$
\mathbf{v} = \frac{1}{T}\sum_{t=1}^{T}\mathbf{A}_t, \quad \mathbf{v} \in \mathbb{R}^{B \times H_3 \cdot d}. \tag{5}
$$

The final classification is obtained by passing the pooled feature vector through a fully-connected layer:

$$
\hat{\mathbf{y}} = \mathbf{W}_{\text{fc}}\mathbf{v} + \mathbf{b}_{\text{fc}}, \quad \hat{\mathbf{y}} \in \mathbb{R}^{B \times N}, \tag{6}
$$

with $N = 10$ being the number of classes. The model's parameters are optimized using the cross-entropy loss function defined as:

$$
\mathcal{L} = -\sum_{i=1}^{B}\sum_{j=1}^{N} y_{ij}\log(\hat{y}_{ij}). \tag{7}
$$

In order to determine the optimal choice of the RNN layers sizes and further fine-tune the model, I relied on a Neural Architecture Search (NAS), which has become an essential tool in designing DL models, particularly when there is a need to balance model performance with computational efficiency (Elsken et al., 2019). The optimal number of neurons in each RNN layer were chosen from a candidate set {128, 256, 512}, a reasonable search set that takes into

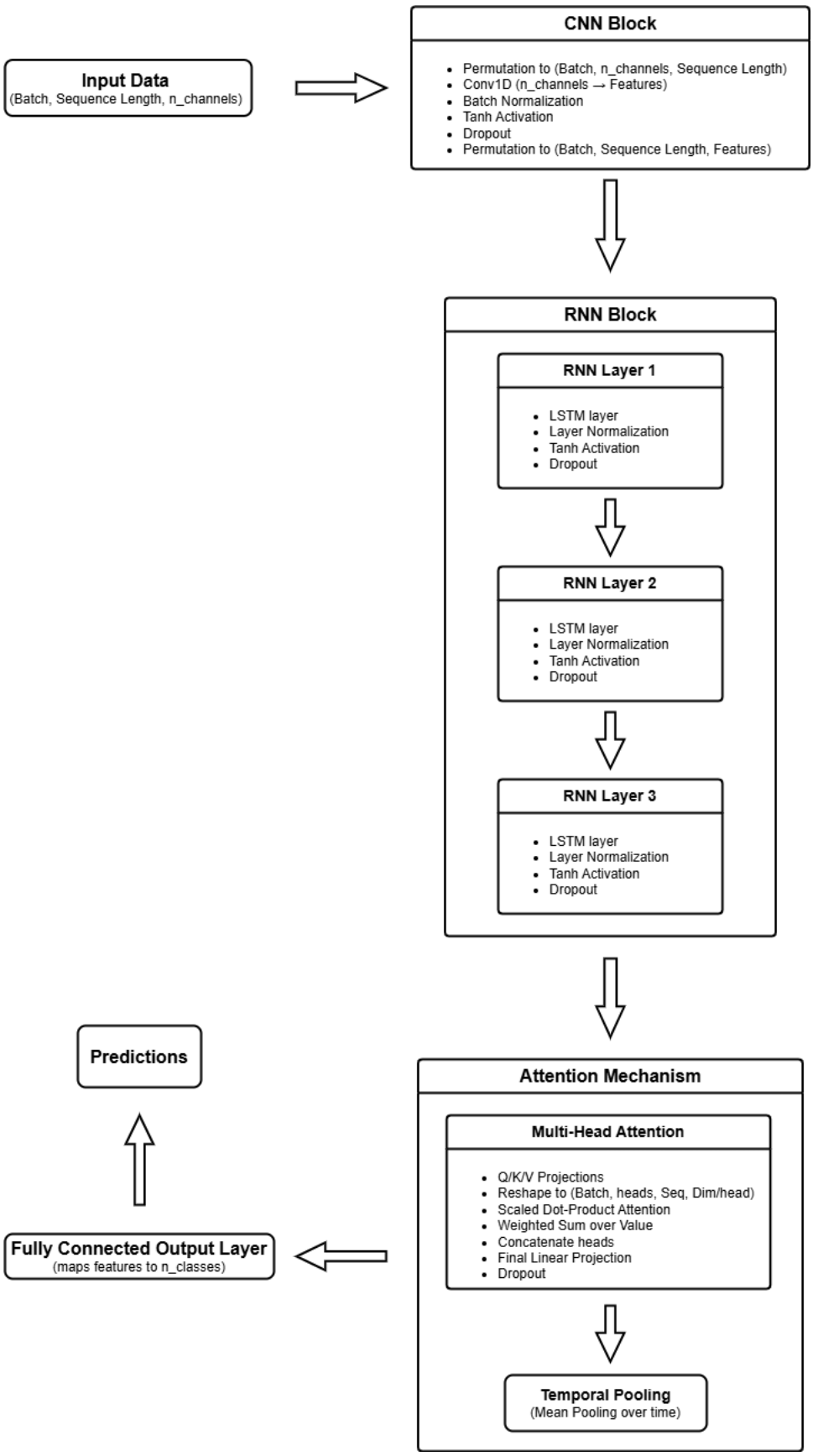


Figure 4: RNN-based model structure.

account both model performance and computational cost. Larger more complicated models can potentially give higher predictive accuracy, but they may also introduce significant

computational efforts, which will result in longer training times or greater resource usage (which is not usually free in cloud computing). This is not practical for resource-constrained applications that value speed and simplicity.

The NAS process is implemented here in the form of a manual grid search over all possible combinations of the hidden sizes for the three RNN layers from the chosen set of possible values. For each candidate architecture, the model is instantiated and then trained using the standard training procedure. Following that, the network is evaluated on the test set using the four basic performance metrics chosen for the previous model: accuracy, F1 score, recall, and precision; as well as execution time (training time). To provide a comprehensive evaluation that encompasses both model effectiveness and efficiency, and that uses all available metrics, I chose a custom composite scoring metric defined by the following formula:

$$S = \sum_{i=1}^{4} w_i m_i - \alpha \frac{T}{3600}, \tag{8}$$

where m_i represents the individual performance metrics (in $[0,\ 1]$), and w_i are the corresponding weights for each metric. The weights were manually chosen as follows: $w_1 = 0.3$ (accuracy), $w_2 = 0.3$ (F1 score), $w_3 = 0.2$ (recall), and $w_4 = 0.2$ (precision). The term T represents the total execution time measured in seconds, and the scaling factor α adjusts the penalty imposed by longer training times. This custom score is designed to give a well-balanced measure of model efficiency where higher performance across the relevant metrics is undeniably favored, yet models that require excessive computational time are penalized proportionally. Dividing the time T by 3600 normalises it into hours, making the penalty term interpretable. In practice, a multiplicative factor of 100 is usually added at the end to scale the final score as a percentage.

The NAS procedure works as follows: for each configuration defined by the triplet (H_1, H_2, H_3), the network is trained using the established training code, and then the various performance metrics are recovered from the results of the test set. After, using the custom score S, it became more intuitive to quantitatively compare the candidate architectures. The grid search iteratively explores the 27 possible configurations, and the configurations showing the highest scores are selected as possible optimal network architecture. Although the exact values of the weights and the penalty factor were determined empirically, they are chosen to balance between maximizing classification performance and minimising computational costs; Accuracy and F1 score are more relevant than precision and recall for model performance, so they received higher weights. The value of $\alpha = 1$ was chosen based on specific examples from

my own tests with the alpha band of the cube dataset. I wanted a value of α that favours a model with a test accuracy of 55% and an execution time of around 200 s, over another with a higher accuracy, 59%, but double the execution time at around 400 s. Choosing the best architecture does not, in fact, fully rely on the results of the aforementioned NAS process, because accuracies vary with each training, therefore some models might over-perform or under-perform during the NAS, which will not properly reflect their abilities. The objective of the NAS was to eliminate the clearly incompatible architectures and select among them the top candidates to consider. Then, more qualitative tests will determine which of those to choose, as it is much simpler to manually compare 3 or 4 models, then to compare all the 27 possible ones. It also depends on the priorities of the user, whether he prefers balance, or puts an emphasis on accuracy, or on speed. For this study, I chose the configuration $(H_1, H_2, H_3) = (128, 256, 128)$ based on its custom score , highest score, and several tests with the dataset. It is also possible to consider the configuration $(H_1, H_2, H_3) = (128, 512, 256)$ since it presented the highest accuracy value if accuracy is the primary concern. The NAS results are available in the appendix section A for further evaluations (for the cube dataset, alpha band).

In order to be incorporated with the proposed RNN-based model efficiently, the EEG data needed to be reshaped into an adequate format that matches its requirements: (*n_sequences*, *sequence_length*, *n_channels*). As seen in the previous discussion about data preprocessing, the data was already separated into ordered temporal sequences, the sequences shuffled among each other, and all that was left was stacking these sequences across a third dimension, the number of sequences. This was simply achieved with the following commands:

```
# Reshape for the RNN
X_shuffled = X_shuffled.reshape(-1, sequence_length, n_channels)
y_shuffled = y_shuffled.reshape(-1, sequence_length)
# Each sequence belongs to one brightness class
# We need 1 prediction for every sequence
y_shuffled = y_shuffled[:, -1]
```

The *sequence_length* = 100 was fixed, similarly to the previous ANN model, for a fair comparison. Although, the batch size is no longer the length of the temporal sequence, but the number of sequences taken with a fixed length of 100. Therefore, a batch would have the following shape: (*batch_size*, *sequence_length*=100, *n_channels*=31). This reshape reduces the

number of steps per epoch compared to the ANN, depending on the choice of batch size.

Following many experiments, the training options were empirically chosen as follows:

- Batch size: 32. It represents the number of temporal sequences processed at every step.
- Optimiser: RMSprop.
- Number of epochs: 50.
- Learning rate: using a scheduler, $1e-4$ until epoch 35, then it gets divided by two, and at epoch 40 it gets divided by 2 again.

Gradient clipping was also used to avoid exploding gradients and sharp changes in accuracy which often occur during the later epochs. This model has $2,254,603$ trainable parameters.

2.2.3 Convolutional Neural Network (CNN): CNN-Transformer model

Convolutional Neural Networks (CNNs) have demonstrated remarkable success in various domains, especially when it comes to processing grid-like data such as images, by automatically learning hierarchical spatial features through numerous layers of convolutions and pooling operations. Given that EEG data can be represented as a spatio-temporal grid (channels and time points), CNNs present a promising approach for analysing brain activity and classifying it into interpretable brain states. Although, standard CNN architectures may not be optimally built to deal with EEG signals specifically.

To address this, some specialised CNN architectures have been developed (Schirrmeister et al., 2017). Among them, one of the most reliable is **EEGNet**, introduced in the paper of Lawhern et al. (2018), a widely used compact CNN designed specifically for EEG-based BCIs. This model tries to capture the most important neurophysiological patterns efficiently by using specific convolutional structures. This network's design usually includes an initial temporal convolution to learn frequency-related filters, followed by a depthwise spatial convolution that learns spatial filters specific to each temporal filter. Following it, a separable convolution block combines depthwise temporal convolutions with pointwise convolutions to efficiently summarise the extracted features and capture higher-level temporal relationships while reducing significantly the number of parameters used compared to standard CNNs.

In this study, building upon the existing EEGNet model, I will test a custom CNN architecture compatible with our task of interest, the brightness level classification, using previously processed EEG data, organised into batches of shape $(batch_size, sequence_length, n_channels)$.

The model I'm proposing here is a standard EEGNet, followed by a transformer block, and then a final linear layer. It adapts the core EEGNet structure, a solid proven model for EEG classification, while adding some enhancements intended to potentially improve performance. The input EEG sequences are first processed by a temporal convolution layer using $F1$ filters of a specified *kernel_length* to capture time-domain patterns. This is immediately followed by a depthwise spatial convolution layer, applying D spatial filters per temporal feature map across all input channels, effectively learning channel-specific spatial patterns associated with the initial temporal features. The output features are then passed through batch normalisation, an activation function (ELU usually), average pooling (*pool_size_1*) to reduce temporal resolution, and dropout for regularisation. The second main block employs a separable convolution, consisting of a depthwise temporal convolution (with *separable_kernel_length*) applied independently to each feature map from the previous block, followed by a pointwise convolution that mixes features across channels, projecting them onto $F2$ output feature maps. This block also includes batch normalisation, activation, average pooling (*pool_size_2*) for further temporal downsampling, and dropout. These convolutional blocks are designed to efficiently extract useful spatio-temporal features from the EEG signals.

The output from the EEGNet, of shape $(batch_size,\ F2,\ 1,\ reduced_sequence_length)$, is then prepared for the Transformer component. The features tensor is squeezed on the second dimension and permuted on the last two dimensions to obtain a new shape compatible with the Transformer's input, $(batch_size,\ reduced_sequence_length,\ F2)$, $F2$ being the embedding dimension. After adding positional encoding to incorporate sequence order information, the tensor of feature vectors is then passed into a standard Transformer encoder block that follows the original design from the paper of Vaswani et al. (2017). This block consists of a Multi-Head Self-Attention layer, followed by a dropout, a residual connection, and layer normalisation. Following that, a position-wise Feed-Forward Network (FFN) with GELU activation and dropout is applied, also followed by a residual connection and layer normalisation.

The self-attention mechanism allows the model to weigh the importance of different time steps in the feature sequence relative to each other, computing these attention scores across multiple "heads" in parallel. Each head operates on a different subspace of the feature representation, potentially capturing diverse aspects of the temporal relationships. It processes the input sequence using learnable Query, Key, and Value projections, computes scaled dot-product attention scores independently for each head, and combines the outputs from all heads before a final linear projection (a more detailed explanation of the attention

module can be found in the following section 2.2.2). The FFN further processes these features independently at each time step. The residual connections and layer normalisations are essential to stabilise the training of deep Transformer models. Finally, the output sequence from the Transformer block is aggregated by performing mean pooling across the time dimension, resulting in feature tensor of shape $(batch_size,\ F2)$, containing a single feature vector of size $F2$ for each sample in the batch. The output sequence from the Transformer block, containing contextually-rich feature vectors, is then fed into a final classification layer to output classification probabilities (logits to be specific) for the $n_classes = 10$ brightness levels. This proposed EEGNet-Transformer architecture, with 3,478,474 trainable parameters, aims to combine the strengths of both models in order to improve performance compared to the base model.

The training options are similar to those of the hybrid RNN, fixed as follows:

- Batch size: 32.
- Optimiser: Adam.
- Number of epochs: 60.
- Learning rate: $1e-4$.

2.3 SHAP (Shapley Additive exPlanation)

The Shapley value is a concept that comes from cooperative game theory (Shapley et al., 1953), it measures the contribution of individual neurons or layers in neural networks. In this cooperative game, neurons are modeled as players and a characteristic function $v(\mathcal{S})$ is used to measure the output of the network when a subset of neurons $\mathcal{S} \subseteq \mathcal{N}$ is active only. Considering the set $\mathcal{N} = \{h_1, h_2, \ldots, h_n\}$ of neurons in the final layer $f_{\text{IN}}(\cdot)$, the payoff for a coalition $\mathcal{S}$ is defined as follows:

$$v(\mathcal{S}) = f_{\text{OUT}}(\mathcal{S}), \tag{9}$$

Where inactive neurons are masked (set to zero for example). The Shapley value ϕ_i^{Sh} for neuron i is computed as its average marginal contribution across all permutations of $\mathcal{N}$:

$$\phi_i^{Sh} = \frac{1}{|\Pi(\mathcal{N})|} \sum_{\pi \in \Pi(\mathcal{N})} [v(\mathcal{P}_i^{\pi} \cup \{i\}) - v(\mathcal{P}_i^{\pi})] \tag{10}$$

$\mathcal{P}_i^{\pi}$ represents the predecessors of i in permutation π. Unfortunately, the computation of the exact formula is almost unfeasible when we consider large networks due to the factorial

complexity, therefore approximations are required. ϕ_i^{Sh} can be estimated using Monte Carlo sampling. This method averages contributions over k randomly sampled permutations:

$$\widehat{\phi}_i^{Sh} \approx \frac{1}{k} \sum_{j=1}^{k} [v(\mathcal{P}_i^{\pi_j} \cup \{i\}) - v(\mathcal{P}_i^{\pi_j})] \tag{11}$$

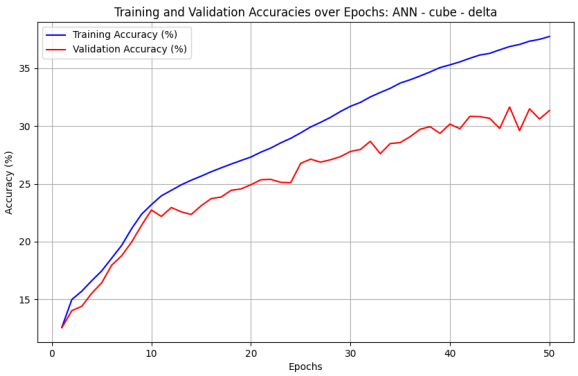
To improve the precision of Shapley value approximations, variance reduction techniques like antithetic sampling (using both a permutation and its reverse) and stratified sampling (grouping permutations with similar marginal contributions) are used. Model-specific methods, such as DeepSHAP, integrate Shapley values with backpropagation rules for more efficient computation, while integrated gradients approximate contributions by integrating gradients along a path from a baseline input. For convolutional or attention layers, structured masking schemes use spatial or hierarchical dependencies to reduce the number of coalition evaluations, making the computation more accessible and more efficient (Lundberg and Lee, 2017; Rozemberczki et al., 2022).

The present research will use the SHAP (SHapley Additive exPlanations) library in Python, introduced by Lundberg and Lee (2017), which provides an efficient implementation of Shapley values for neural networks and other machine learning models. It supports various approximation methods, like KernelSHAP and DeepSHAP, which are prepared and optimised for different architectures. When it comes to neural networks, SHAP library uses gradient-based techniques to compute Shapley values efficiently, enabling explanations and feature importance rankings. While computational cost and interpretability limitations remain as challenges, this library still offers a practical and useful tool to apply Shapley values on ML models.

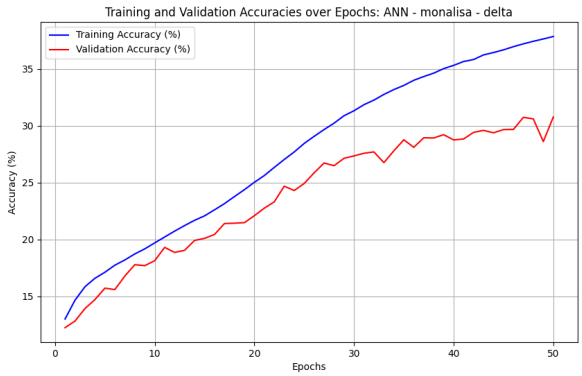
3 Results and analysis

3.1 Qualitative analysis of the ANN model performance

An evaluation of the base ANN model's performance for classifying visual brightness levels based on EEG data was conducted by training it over the **Necker Cube** and **Monalisa** datasets, across all the frequency bands (The model was trained separately for every frequency band from the 2 dataset). Here are the accuracy curves obtained during the training process for this model:

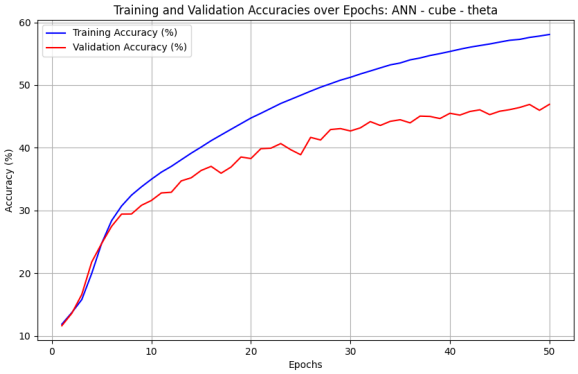


(a) Cube dataset.

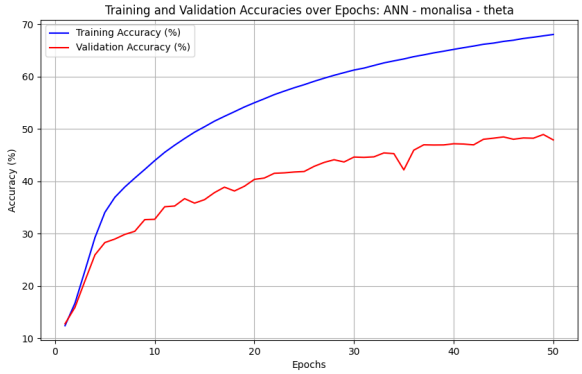


(b) Monalisa dataset.

Figure 5: Delta band accuracy.

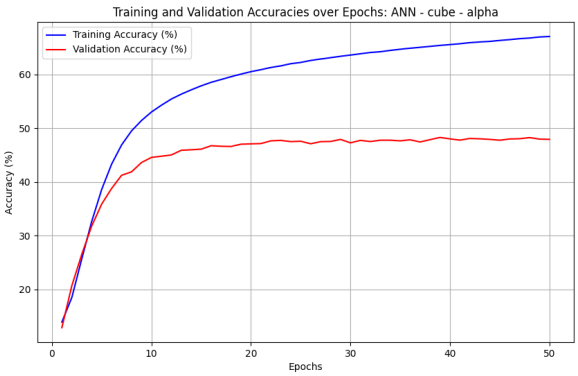


(a) Cube dataset.

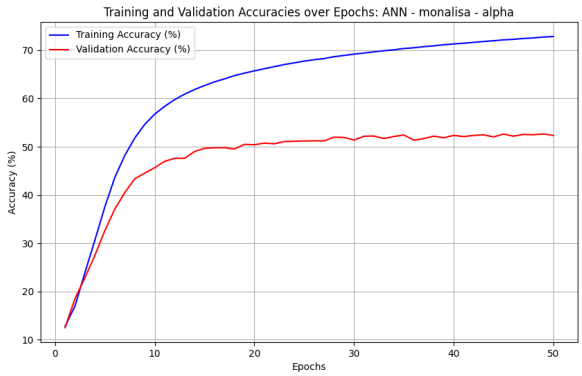


(b) Monalisa dataset.

Figure 6: Theta band accuracy.

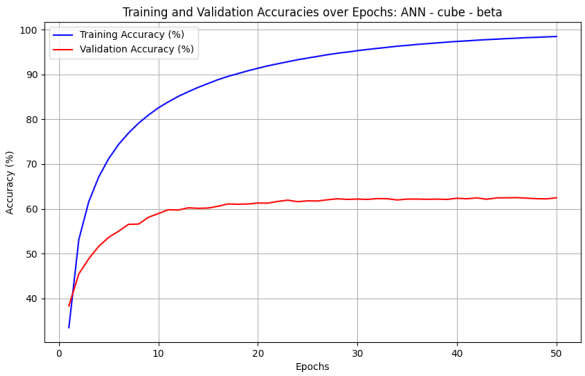


(a) Cube dataset.

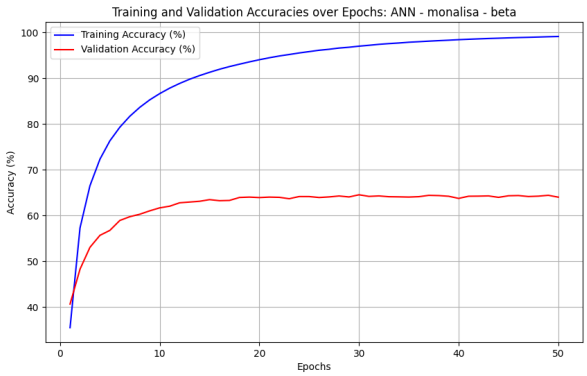


(b) Monalisa dataset.

Figure 7: Alpha band accuracy.

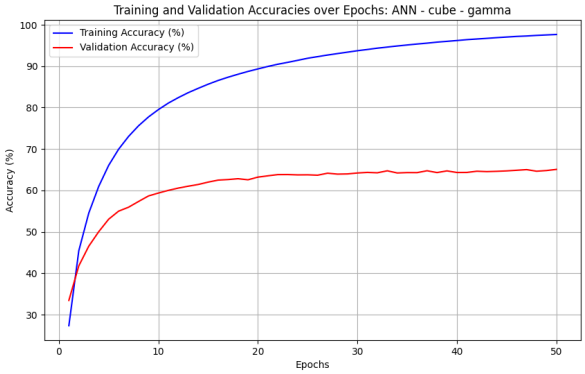


(a) Cube dataset.

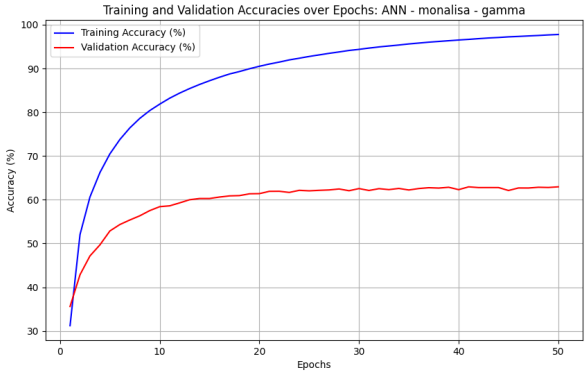


(b) Monalisa dataset.

Figure 8: Beta band accuracy.



(a) Cube dataset.



(b) Monalisa dataset.

Figure 9: Gamma band accuracy.

The analysis considered the learning dynamics revealed by both the training and validation accuracy curves over 50 epochs, as well as the final quantitative metrics obtained on the unseen test set (tables 1 and 2). The examination of these learning curves consistently shows that the training accuracy increases steadily throughout the epochs for all datasets and frequency bands, indicating the model's capacity to learn representations from the training data. This learning is typically characterized by a rapid improvement phase during the initial epochs (approximately 0-15), followed by a more gradual ascent towards higher accuracy levels.

However, the validation accuracy curves present a deeper picture of the model's generalisation capabilities. While validation accuracy generally increases in the early training stages, reflecting the continuous training progress, it tends to plateau significantly earlier, often between 15 and 25 epochs. Beyond this point, further training gives diminishing accuracy rewards and negligible improvements, or sometimes even some slight fluctuations

in validation performance. The divergence between the continuously increasing training accuracy and the saturating validation accuracy is a prominent feature across nearly all experimental conditions, serving as a strong indicator of **overfitting** (Ying, 2019). This suggests that the ANN model becomes overly specialized to the randomness and noise within the training dataset, hindering its ability to generalise effectively when it comes to unseen data such as the validation set.

The final test results further confirm the limitations observed during validation. The test accuracies reported in tables 1 and 2 align in general with the 'plateau' levels reached by the validation curves, confirming that the overfitting observed during training translates into limited performance on the independent and unseen test data. Performance varies considerably across frequency bands as can be seen; First, test accuracies greatly differ between frequency bands, but this could be attributed more to the nature of each frequency band rather than to the model itself and its inner architecture. Secondly and more importantly, for both datasets, the beta and gamma bands achieve the highest test accuracies (around $63 - 65\%$), yet their corresponding learning curves exhibit a high overfitting level, with training accuracies soaring far beyond the validation/test levels and approaching the perfect score of 100%. The alpha and theta bands give moderate test accuracies, ranging from approximately 44% to 52%. They also present noticeable but slightly less severe overfitting gaps compared to the higher frequency bands. Consistently, the delta band demonstrates the lowest performance, with test accuracies around 32% for both datasets. While the overfitting gap between training and validation accuracies is less present for the delta band, the overall low accuracy indicates that the delta band holds less distinctive information relevant to this particular classification task. This interpretation is supported by the nature of this band, as mentioned in the datasets section 2.1.1, that makes it mostly irrelevant when it comes to conscious and cognitively-demanding tasks. Finally, comparing the datasets between each other, performance trends seem to be largely similar, although the **Monalisa** dataset shows slightly better accuracy values in the theta and alpha bands compared to the **Cube** dataset. This is likely related to the nature of the visual stimuli used in these datasets.

In summary, while the standard baseline ANN model demonstrates the ability to learn from the EEG data, its performance is significantly hindered by overfitting, particularly in the richer and more informative beta and gamma bands. This limits its generalisation ability and motivated the exploration of alternative architectures which can better capture temporal dependencies within the EEG signals and potentially improve, not only the results for this classification task, but also the computational times as it seems the ANN architecture used

needs to compensate its incompatibility with the EEG data by adding more complexity to its network.

Table 1: Test results for the Cube dataset with the base ANN model

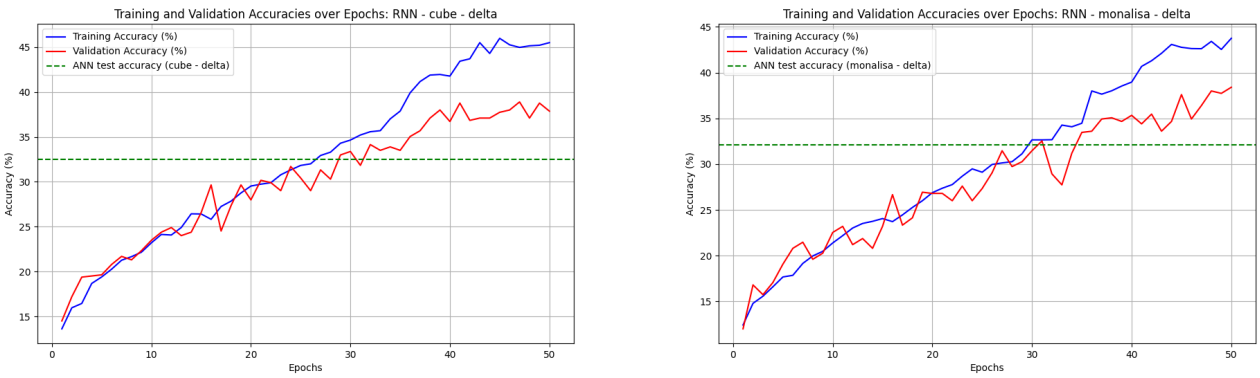
Band	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	Training time (s)
Delta	32.477	32.591	33.139	32.477	2049.7
Theta	44.401	44.553	44.595	44.438	2054.2
Alpha	48.742	49.005	48.783	48.736	2045.4
Beta	63.203	63.092	63.070	63.033	2041.1
Gamma	63.093	63.228	63.170	63.118	2052.8

Table 2: Test results for the Monalisa dataset with the base ANN model

Band	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	Training time (s)
Delta	32.120	31.979	32.341	31.877	2005.0
Theta	49.348	49.565	49.497	49.316	1898.4
Alpha	52.079	51.753	51.682	51.677	1885.7
Beta	64.501	64.265	64.439	64.260	1892.3
Gamma	63.237	63.299	63.162	63.121	1899.2

3.2 Qualitative analysis of the hybrid RNN model performance

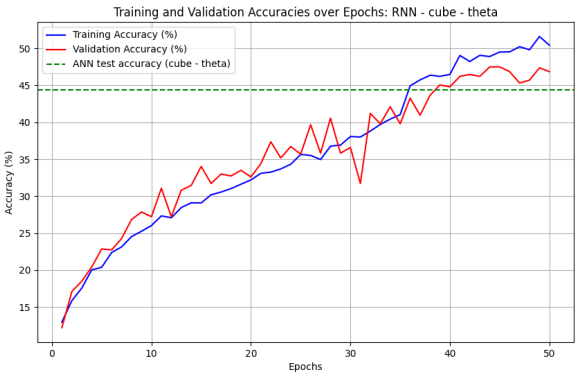
Following the evaluation of the baseline ANN model, I turn my attention now to the proposed hybrid RNN model, which incorporates with its core LSTM layers a CNN element (a CNN frontend to be specific) and attention mechanisms. This analysis aims to evaluate the learning characteristics and highlight the performance of the RNN model on the unseen test data and, ultimately, to compare its efficiency against the previously established ANN model for the task of EEG-based brightness level classification across the **Cube** and **Monalisa** datasets.



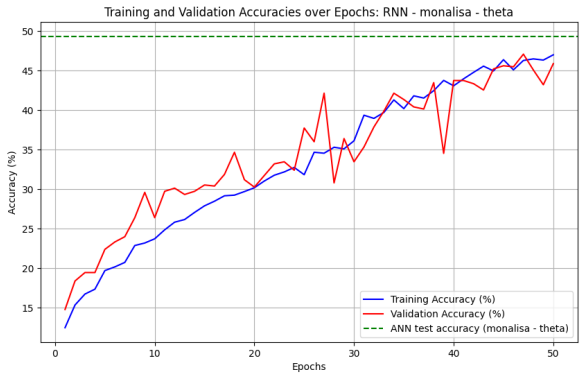
(a) Cube dataset.

(b) Monalisa dataset.

Figure 10: Delta band accuracy.

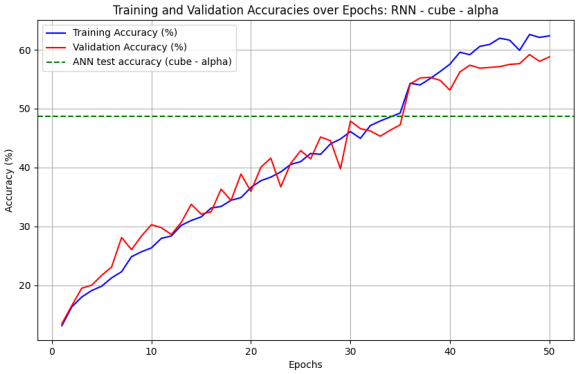


(a) Cube dataset.

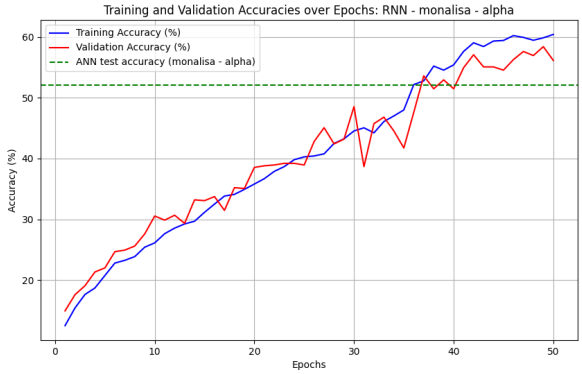


(b) Monalisa dataset.

Figure 11: Theta band accuracy.

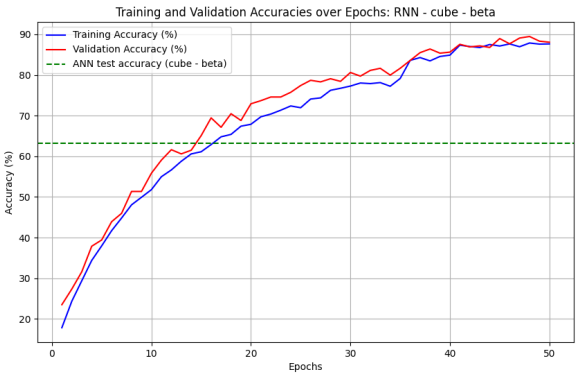


(a) Cube dataset.

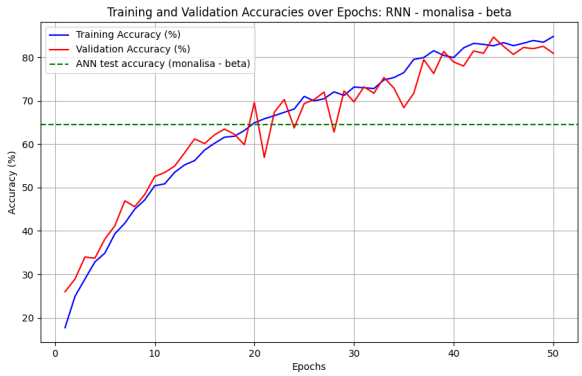


(b) Monalisa dataset.

Figure 12: Alpha band accuracy.

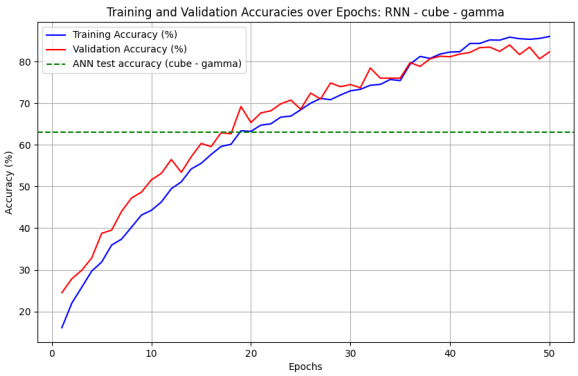


(a) Cube dataset.

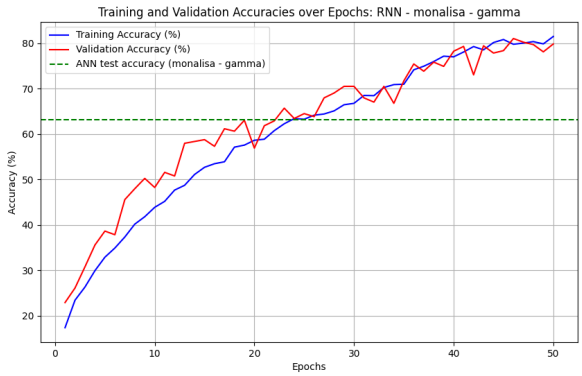


(b) Monalisa dataset.

Figure 13: Beta band accuracy.



(a) Cube dataset.



(b) Monalisa dataset.

Figure 14: Gamma band accuracy.

The fundamental difference in data handling is notable, as mentioned in 2.2.2: the ANN, on one hand, processed one feature vector from each 100-sample batch independently, of shape (*batch_size*, *n_channels*), potentially losing temporal context within the batch; on the other hand, the hybrid RNN processes data in sequences of 100 time steps, of shape (*batch_size*, *sequence_length*, *n_channels*), explicitly designed to capture temporal dependencies using LSTM layers, complemented by its convolutional front-end and its attention mechanism.

The examination of the training and validation accuracy curves for the RNN-based model reveals some distinct learning dynamics compared to the ANN model. While both training and validation accuracies generally show an upward trend over the 50 epochs, unlike previously with the ANN, a key difference emerges in the relationship between these two curves. For the RNN, the gap between the training and validation accuracy is clearly narrower across most frequency bands and both datasets compared to the significant divergence seen with the ANN. This suggests a substantial reduction of the overfitting problem that severely hindered the ANN. The RNN's validation accuracy tends to track the training accuracy more closely, indicating better generalisation during the learning process. Although, these particular validation curves display more fluctuations during training, particularly in later epochs, likely reflecting the stochastic nature of mini-batch optimisation with a more complex, recurrent DL architecture. But they do not in general show the early saturation of the validation accuracy observed with the ANN.

A direct visual comparison became easier by including the ANN's final test accuracy as a dashed green line on the RNN learning plots. In numerous cases, particularly for the beta and gamma bands which were the best performing for the ANN, both the training and validation accuracies of the RNN model surpass the ANN's final test performance level

relatively early in the training, often well before 20-30 epochs. For instance, observing the **cube-beta** plot, the RNN's validation accuracy consistently stays above the ANN's 63.2% test benchmark after approximately epoch 15 and continues to climb towards significantly higher values. This dynamic shows a clear and sharp contrast with the ANN's performance, where validation accuracy 'plateaued' near its eventual test result while training accuracy continued to diverge upwards. The reduced overfitting in the RNN model allows it to continue improving its generalisation capability for a longer duration of the training process.

The quantitative results obtained on the test sets tables 3 and 4) provide definitive evidence of the RNN model's superior performance. Across virtually all frequency bands and both datasets, the RNN achieves substantially **higher test accuracies** compared to the ANN (tables 1 and 2). The improvements are particularly high in the beta and gamma bands. For the **Cube** dataset, beta accuracy surged from 63.2% (ANN) to 84.7% (RNN), representing an improvement of over 0.21 accuracy value. Similarly, gamma accuracy increased from 63.1% to 82.9%, a gain of nearly 0.2 accuracy value. Comparable significant improvements are also observed for the **Monalisa** dataset in these bands: beta: 64.5% to 82.4%; and gamma: 63.2% to 84.3%. Another important detail observed, the RNN also elevates performance in the bands where the ANN previously struggled. For the **cube** dataset, alpha band accuracy improved from 48.7% to 61.4%, theta from 44.4% to 49.4%, and even the challenging delta band experienced an increase from 32.5% to 43.9%. Although, the RNN got close but failed to surpass the ANN accuracy for the monalisa dataset with the theta band (49.35% vs 48.27%). These noticeable gains in accuracy were also supported by improvements in the other used metrics as well; Recall, Precision, and F1-score; Thus, further highlighting the RNN's enhanced ability to classify brightness classes based on recorded EEG signals. Furthermore, these performance enhancements are accompanied by a **significant reduction in training time**, with the RNN model completing training roughly 7-8 times faster than the ANN (approximately 260-270 seconds versus over 2000 seconds).

In conclusion, the analysis of learning curves and test metrics clearly demonstrates the advantages of the hybrid RNN architecture over the standard ANN for this specific EEG classification task. The RNN effectively deals with the overfitting issues observed with the ANN, leading to significantly better generalisation and considerably higher classification accuracies across all tested frequency bands and datasets. The ability of the RNN, likely relying on its LSTM components and attention mechanism, to process temporal sequences seems crucial for extracting more discriminative features from the EEG data. This approach not only enhanced performance, but also was achieved with a much greater computational efficiency

during training, therefore strongly supporting the choice of the RNN-based approach to classify brain activity related to visual perception in the context of the current study.

Table 3: Test results for the Cube dataset with the hybrid RNN model

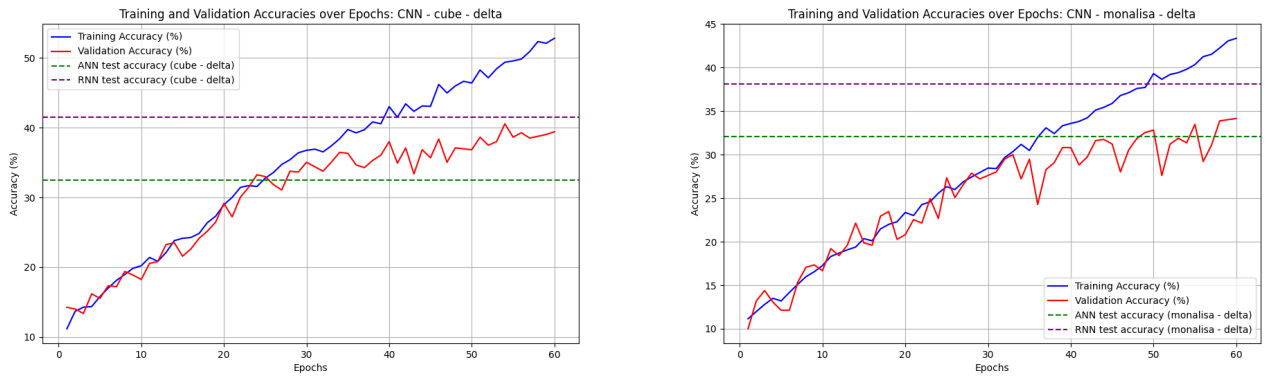
Band	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	Training time (s)
Delta	43.902	43.93	44.254	45.035	268.5
Theta	49.422	49.510	49.239	49.011	269.3
Alpha	61.361	61.301	61.701	61.285	268.0
Beta	84.724	84.903	84.529	84.582	271.2
Gamma	82.927	83.021	83.242	83.068	267.4

Table 4: Test results for the Monalisa dataset with the hybrid RNN model

Band	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	Training time (s)
Delta	38.133	38.101	38.366	37.900	265.0
Theta	48.267	48.459	49.047	48.399	267.4
Alpha	58.933	58.770	61.189	59.323	259.5
Beta	82.400	82.357	82.425	82.159	261.8
Gamma	84.267	84.405	84.210	84.232	263.8

3.3 Qualitative analysis of the CNN-Transformer model performance

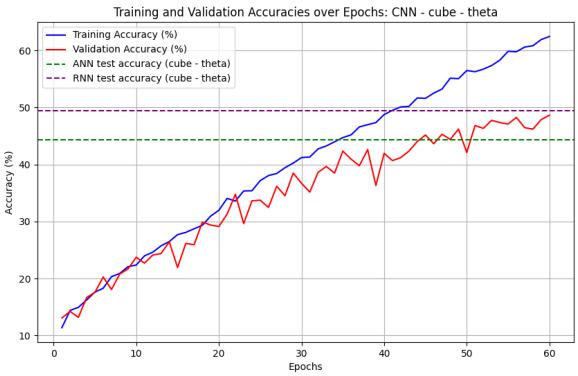
I now turn to evaluating the performance of the hybrid CNN-Transformer model, referred to in code as the class "EEGNet_custom", which integrates the feature extraction capabilities of EEGNet with a Transformer encoder block using positional encoding and multi-head self-attention. This analysis focuses on the training and validation accuracy curves over 60 epochs, alongside the final test metrics (Tables 5 and 6), and provides insight into performance compared to the previously assessed ANN and RNN-based models.



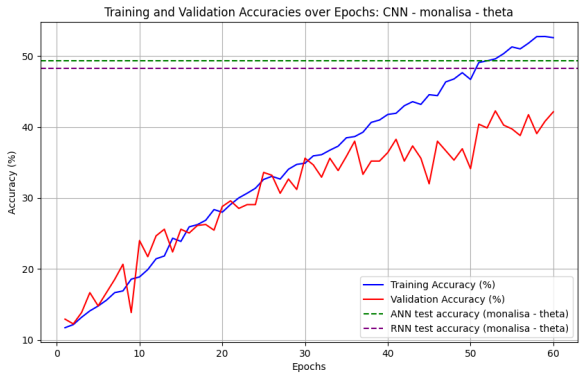
(a) Cube dataset.

(b) Monalisa dataset.

Figure 15: Delta band accuracy.

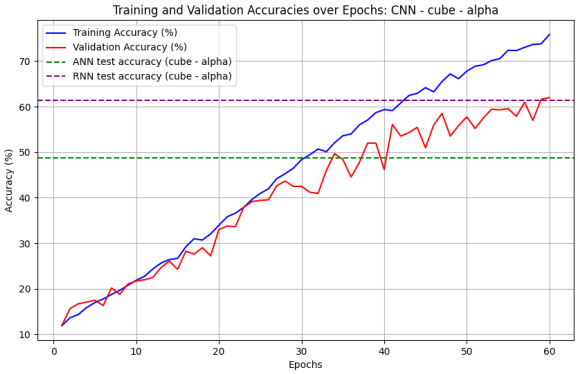


(a) Cube dataset.

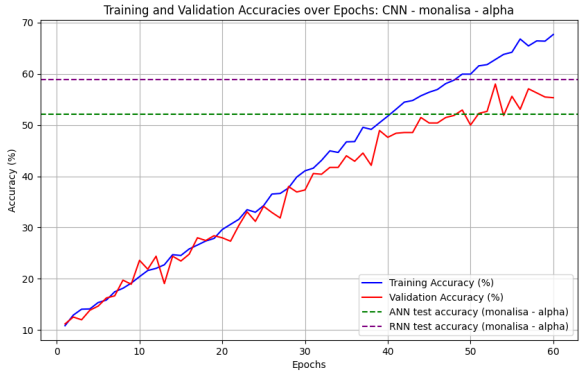


(b) Monalisa dataset.

Figure 16: Theta band accuracy.

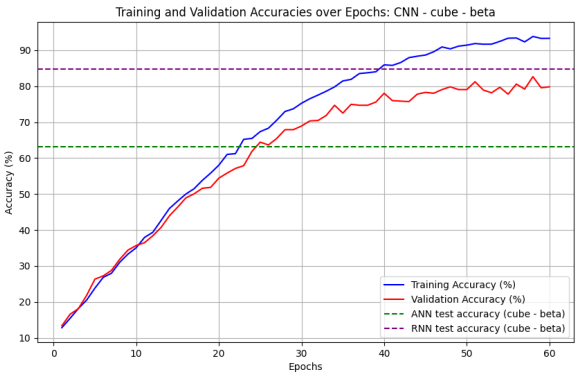


(a) Cube dataset.

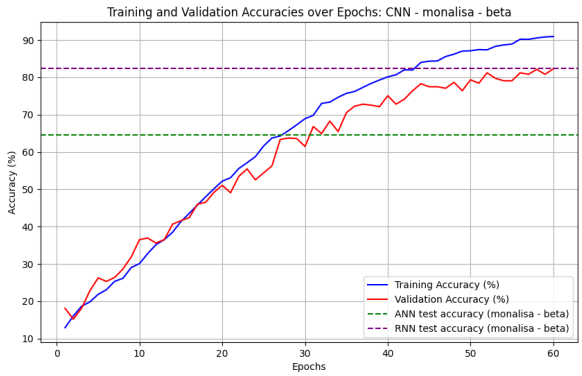


(b) Monalisa dataset.

Figure 17: Alpha band accuracy.

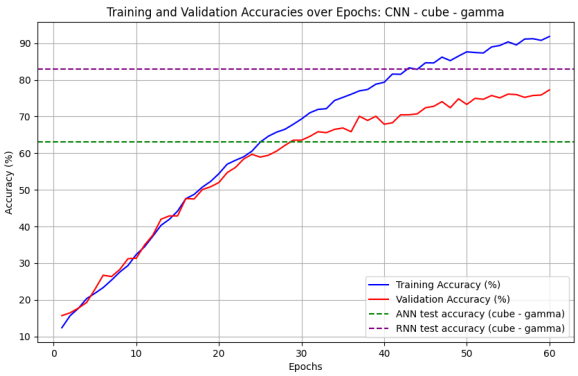


(a) Cube dataset.

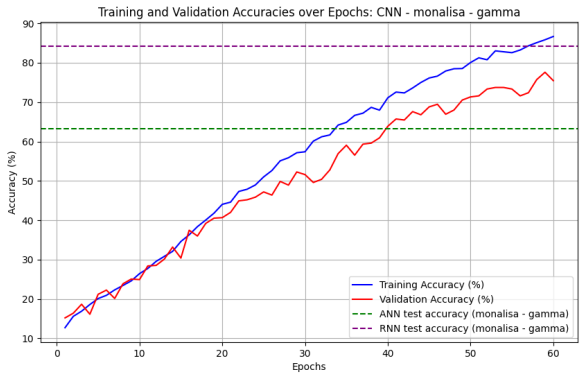


(b) Monalisa dataset.

Figure 18: Beta band accuracy.



(a) Cube dataset.



(b) Monalisa dataset.

Figure 19: Gamma band accuracy.

Observing the accuracy plots, the CNN-Transformer model demonstrates some consistent learning, with both training and validation accuracies generally increasing throughout the training period across both datasets and all frequency bands. A main key observation relative to the baseline ANN is the improved control over overfitting. While a gap between training and validation accuracy still emerges for this model, particularly in later epochs, it is visibly less severe than the significant divergence that appeared in the case of the standard ANN. This indicates that the combined EEGNet feature extraction and Transformer architecture generalises the EEG data better than a simple, although large in size, ANN. However, the validation accuracy curves often demonstrate considerable fluctuations, potentially more so than observed with the RNN model, suggesting some instability or sensitivity during the optimisation process for this more complex architecture.

The validation accuracy of the CNN-Transformer often surpasses the final test performance of the ANN (except for 'monalisa-theta' dataset), and mostly before reaching the latter epochs of the training for most bands. However, when compared to the RNN model, it reveals a different insight. The CNN-Transformer's validation accuracy typically progresses below the RNN test accuracy line throughout the 60 epochs. Only in a couple of specific dataset-band combinations (monalisa-beta and cube-alpha to be specific, towards the very end) does the validation curve approach or slightly exceed the RNN's final test result. This suggests that the CNN-Transformer, while effective, was not able to reach the same level of generalisation as the hybrid RNN during training.

The quantitative test results detailed in Tables 5 and 6 confirm the observations from the training/validation plots. The CNN-Transformer model achieves much higher test accuracies (especially for beta and gamma bands) than the baseline ANN, almost across all

dataset-band pairs (except monalisa-theta). For example, in the cube dataset, beta accuracy improved from the ANN's 63.2% to 81.1%, and gamma from 63.1% to 79.1%. Similar gains are seen in the monalisa dataset, particularly for the higher-frequency bands (alpha, beta and gamma). These results confirm the general improvement relative to the ANN. However, compared to the hybrid RNN's test results (Tables 3 and 4), the CNN-Transformer generally falls short of achieving the peak performance levels reached by the RNN. This is particularly true in the Beta and Gamma bands (cube-beta: 81.1% vs RNN's 84.7%; monalisa-gamma: 77.7% vs RNN's 84.3%), but also holds for most other bands including Alpha, Theta, and Delta (cube-delta: 40.8% vs RNN's 43.9%). The exceptions where the CNN-Transformer slightly outperformed the RNN were few (monalisa-beta: 82.7% vs 82.4%, and cube-alpha: 63.7% vs 61.4%). When it comes to computational efficiency, the training times for the CNN-Transformer are generally comparable to those of the RNN (around 260-300 seconds), also showing a significant efficiency gain over the ANN (with training times around 2000 seconds).

An additional factor of value here is model complexity from the perspective of the number of trainable parameters. The CNN-Transformer used the most parameters (3,478,474), compared to the ANN (3,208,675) and the RNN (2,254,603). The RNN-based model, on one hand, achieved the best performance with the lowest parameter count among the three, suggesting high efficiency. On the other hand, the CNN-Transformer's increased complexity did not translate into more accuracy gains over the RNN despite its clear efficiency relative to the ANN. This edge the CNN-Transformer has over the ANN is mostly due to its highly optimised blocks that take advantage of the available GPU well.

To conclude, the CNN-Transformer model represents a significant improvement over the baseline ANN, effectively taking advantage of both convolutional feature extraction specifically optimised for EEG data (via EEGNet) and sequence processing capabilities (through the Transformer encoder block) to achieve higher accuracy, better generalisation, and higher computational efficiency (similar to the RNN). It also succeeded at limiting the overfitting of the ANN, although still slightly present. However, it did not consistently surpass the peak performance achieved by the hybrid RNN, particularly in the most relevant frequency bands containing the highest feature-related information (beta and gamma bands, most active during cognitively-demanding tasks). This suggests that while the EEGNet-Transformer pair is quite powerful, the specific combination of the frontend-CNN, LSTM layers and self-attention module in the hybrid RNN model is still slightly better at capturing the most critical temporal dependencies within the studied EEG datasets for brightness-level

classification, leading to its superior classification results. The CNN-Transformer model, nonetheless, is still a highly effective and efficient architecture for this specific EEG analysis.

Table 5: Test results for the Cube dataset with the CNN-Transformer model

Band	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	Training time (s)
Delta	40.822	41.220	41.748	40.854	268.0
Theta	47.754	47.662	48.330	47.570	270.4
Alpha	63.671	63.705	64.320	63.400	267.9
Beta	81.130	81.086	81.261	81.039	269.1
Gamma	79.076	78.828	79.070	78.650	302.9

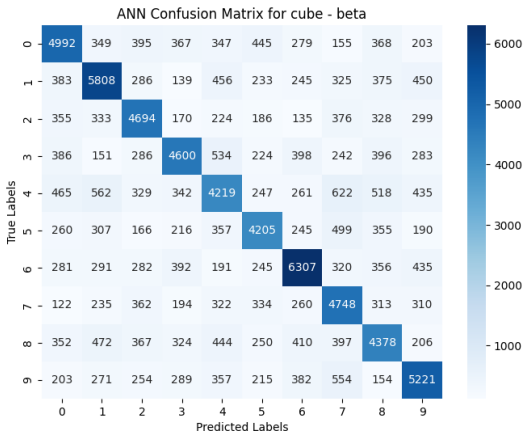
Table 6: Test results for the Monalisa dataset with the CNN-Transformer model

Band	Accuracy (%)	Recall (%)	Precision (%)	F1 (%)	Training time (s)
Delta	36.267	36.097	37.599	36.243	278.2
Theta	43.200	43.540	45.361	43.461	256.7
Alpha	54.400	54.054	54.801	53.993	259.7
Beta	82.667	82.690	82.487	82.524	257.1
Gamma	77.733	77.591	78.011	77.679	282.1

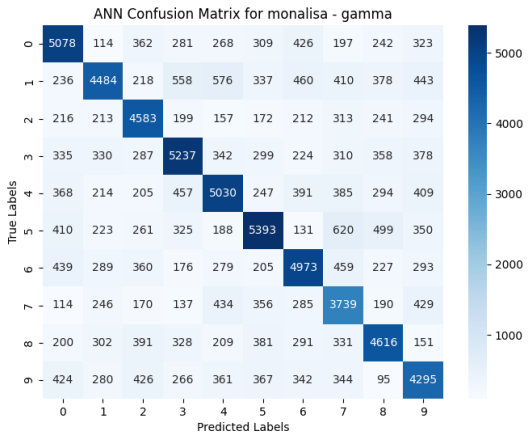
Based on the previous results, which identified the **beta** and **gamma** bands as the most influential for the cognitive task of interest to this work due to the high accuracy the model achieved with them and their theoretical interpretation in the field of neuroscience, I will be focusing my analysis on these two bands from this point onward.

3.4 Analysis of confusion matrices of the top performing frequency bands

To better understand how the models classify brightness levels and where errors arise, I will be examining in more detail a selected set of confusion matrices. The previously discussed results showed that **beta** and **gamma** bands reached the highest accuracies, therefore confirming that they contain the most feature-related information compared to other frequency bands. This analysis will focus in particular on the pairs **dataset-band** that provided the best performances, or the conditions in which the highest accuracy was achieved for each stimulus, using the best overall model (the RNN). Therefore, **cube-beta** and **monalisa-gamma** models were chosen. Comparing the confusion matrices from the ANN, hybrid RNN, and CNN-Transformer for these pairs will give more insights on how different model architectures influence classification accuracy and error patterns when the related brightness neural signal is strongest and most informative.

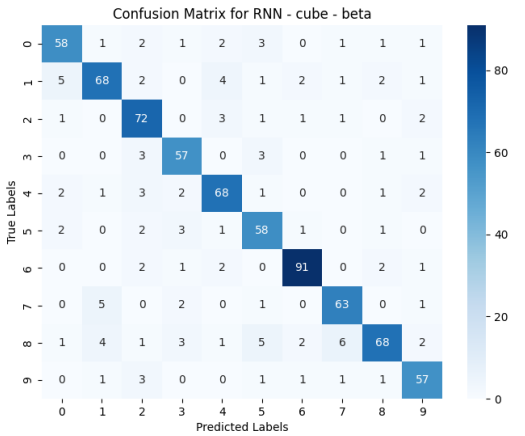


(a) Cube-Beta.

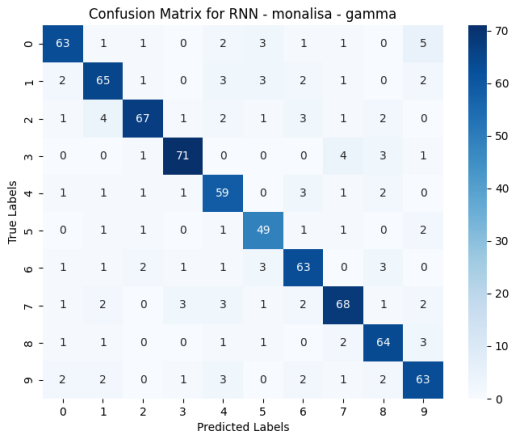


(b) Monalisa-Gamma.

Figure 20: Confusion matrices for the ANN.

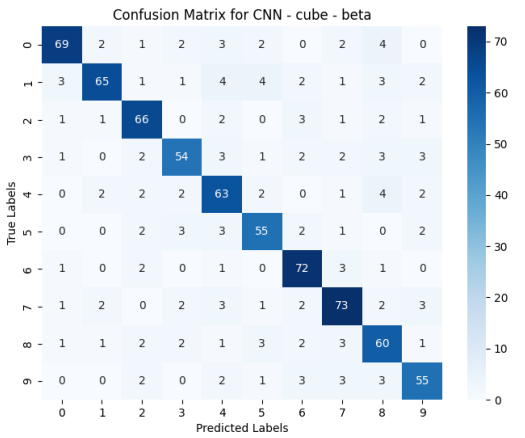


(a) Cube-Beta.

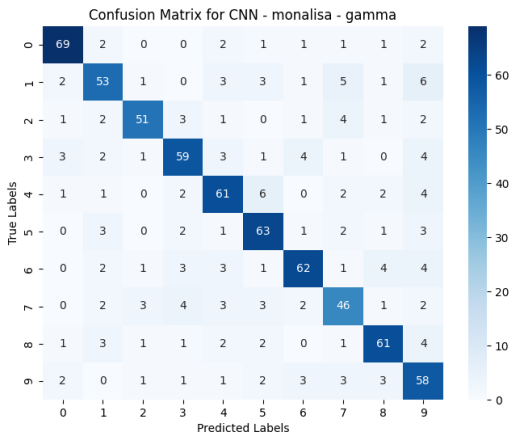


(b) Monalisa-Gamma.

Figure 21: Confusion matrices for the hybrid RNN.



(a) Cube-Beta.



(b) Monalisa-Gamma.

Figure 22: Confusion matrices for the CNN-Transformer.

For the **cube-beta** pair, the ANN's confusion matrix on one hand shows a relatively weak diagonal line, with numerous high counts scattered across non-diagonal cells. This visual representation of classification results reflects frequent misclassifications between brightness classes, relating to the ANN's average accuracy (approximately 63.2%). On the other hand, the RNN matrix for **cube-beta** displays a strong concentration along the main diagonal. Most predictions are correct, and off-diagonal entries are sparse, aligning with the RNN's high accuracy (approximately 84.7%). The errors that do occur seem slightly more frequent between adjacent brightness levels, suggesting that the classification confusion mainly happens between similar levels rather than due to random incorrect assignments. The CNN-Transformer matrix also shows clear improvement over the ANN with a strong diagonal. However, compared to the RNN, it shows a slight increase in counts of non-diagonal cells, also consistent with its achieved test accuracy (approximately 81.1%) being high but a little below the RNN's accuracy.

The analysis of the **monalisa-gamma** pair also shows some similar trends. The ANN matrix is again showing some significant counts for non-diagonal cells, indicating classification confusion and reflecting its limited accuracy (around 63.2%). The RNN matrix, being consistent to its high accuracy (around 84.3%), shows a clearly strong diagonal, indicating effective class separation during classification. Non-diagonal errors are still present, but remained low in counts. The CNN-Transformer matrix shows some good classification quality, easily surpassing the ANN with a clear and well-defined diagonal line. But it falls short when compared to the hybrid RNN model, as it is again showing some slightly higher non-diagonal counts, reflecting its test accuracy of around 77.7% being lower than the RNN's accuracy for this **monalisa-gamma** pair.

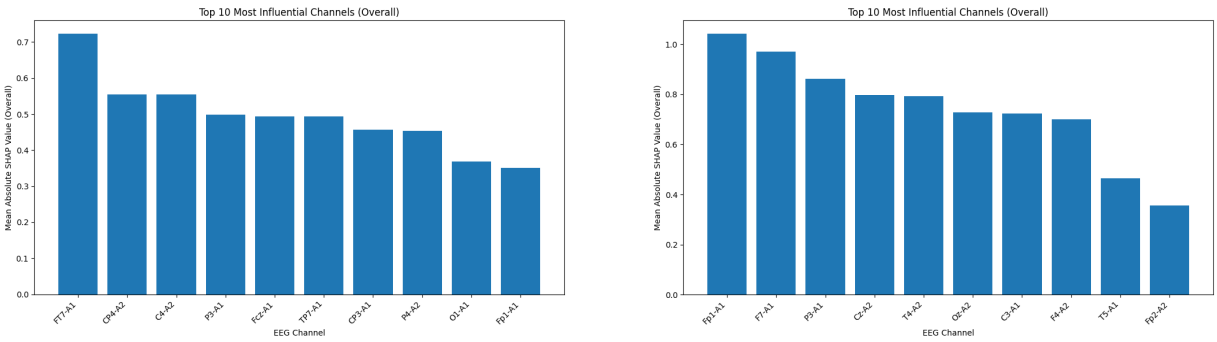
To summarise, the inspected confusion matrices for the **cube-beta** and **monalisa-gamma** pairs visually confirm the quantitative test results obtained previously. Both the RNN and CNN-Transformer architectures achieved considerably better class separation and reduced classification confusion compared to the standard ANN. The RNN model, which attained the highest accuracies for the two chosen band-dataset pairs, presents confusion matrices with the strongest diagonal concentration and the fewest off-diagonal errors. The CNN-Transformer also performed well, exceeding the ANN, although its matrices suggest a less sharp classification than the hybrid RNN.

3.5 SHAP Analysis

3.5.1 Overall feature importance

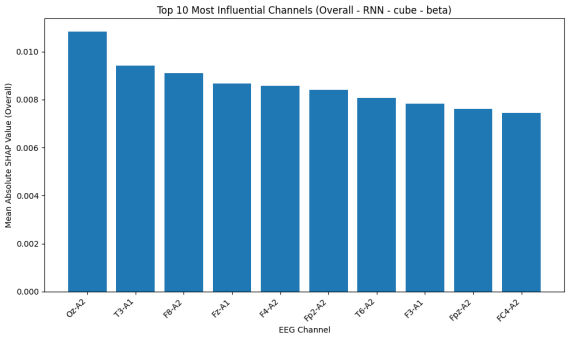
To identify which EEG features most significantly influence the models' predictions for brightness classification, I used SHAP (SHapley Additive exPlanations). This initial SHAP analysis focuses on the overall importance attributed to individual EEG channels by the ANN, RNN, and CNN-Transformer models, providing a high-level view of which brain regions each model prioritized. For this examination, I selected the **cube-beta** and **monalisa-gamma** conditions, as these pairings generally gave the highest classification accuracies with all the models, suggesting they capture particularly informative neural signals related to the brightness perception task. It's important to note that in the channel nomenclature (e.g., Fp1-A1, O2-A2), A1 typically refers to the left hemisphere and A2 to the right hemisphere.

The SHAP values were calculated using the **KernelExplainer** method. For the ANN, which processes each of the 100 time points within a batch as independent features (effectively treating the 0.4-second data segment without explicit temporal dependency), mean absolute SHAP values were calculated directly for each channel input. For the RNN and CNN-Transformer, which are designed to model temporal sequences, SHAP values were first computed for features defined as a specific channel at a particular time step within the analysed sequence segment (first 30 time points only, not the whole 100-point sequence, due to GPU memory constraints on Google colab). To obtain an overall importance score for each channel for these sequence models, I took the absolute SHAP values, averaging them across test samples, summing these averaged values across the time dimension of the sequence segment, and finally averaging these sums across all output classes. The obtained bar plots show the top 10 channels ranked by this calculated mean absolute SHAP value for each dataset-band pair.

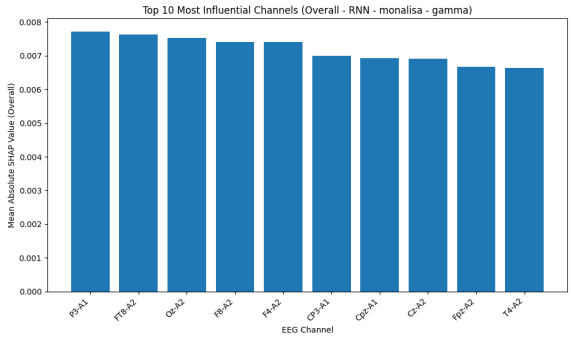


(a) Cube-Beta. (b) Monalisa-Gamma.

Figure 23: Top 10 feature importance for the ANN.

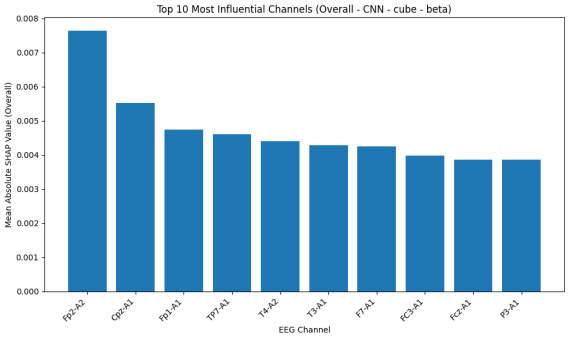


(a) Cube-Beta.

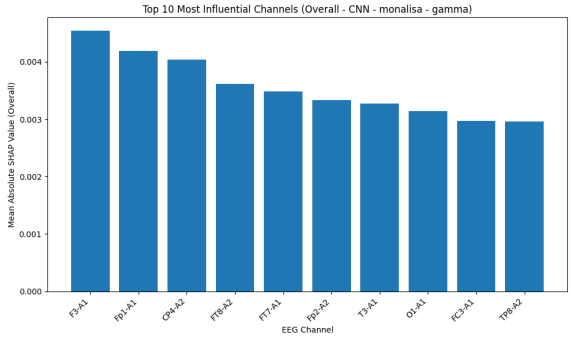


(b) Monalisa-Gamma.

Figure 24: Top 10 feature importance for the hybrid RNN.



(a) Cube-Beta.



(b) Monalisa-Gamma.

Figure 25: Top 10 feature importance for the CNN-Transformer.

A key observation shows when examining the top 10 influential channels for the hybrid RNN model. For both the **cube-beta** and **monalisa-gamma** conditions, a majority of the top 10 channels identified by the RNN (seven out of ten in each case) are located in the right hemisphere (e.g., F8-A2, F4-A2, Fp2-A2 for **cube-beta**; FT8-A2, F8-A2, F4-A2, CP4-A2 for **monalisa-gamma**), with others being midline or left-sided. This pattern of right-hemisphere emphasis by the RNN aligns with the well-established and accepted hypothesis in neuroscience suggesting a right hemisphere dominance when it comes to visual perception tasks (Corballis, 2003; Roser et al., 2011).

The ANN model, on the other hand, due to its processing of individual time points from the short 0.4-second window without direct sequence (or time) processing, identified a more scattered set of important channels for both conditions, with no clear hemispheric dominance consistently showing from its top 10 features. For example, in **cube-beta**, FT7-A1 (left) was most important, followed by a mix of right and left channels. For **monalisa-gamma**, the ANN mainly highlighted left-hemisphere channels like Fp1-A1 and F7-A1.

The CNN-Transformer model presented another distinct pattern. For **cube-beta**, its top channels included Fp2-A2 (right prefrontal) and Cpz-A1 (midline centroparietal), but

also significant left-sided channels like Fp1-A1 and TP7-A1. More surprisingly, for the **monalisa-gamma** condition, the CNN-Transformer attributed high importance especially to channels F3-A1 and Fp1-A1 (left-hemisphere frontal and prefrontal), and also FT7-A1 (left frontotemporal), which is a curious result when compared to the RNN's tendency to the right hemisphere dominance, a known and well-backed hypothesis on visual perception in the field of neuroscience.

The previous analysis of the overall SHAP channel importance leads to the conclusion that the specific EEG channels deemed most critical highly depend on the model architecture and how it processes the input data. The RNN model consistently showed a notable emphasis on right-hemisphere channels in its top 10 features for both examined conditions, which is in accordance with theories of right-hemisphere dominance in visual tasks. The ANN, with its simpler, non-sequential processing of short data segments, showed more varied and less lateralized patterns. The CNN-Transformer, while an advanced sequence model, identified different dominant regions, even pointing to left-hemisphere frontal areas for the **monalisa-gamma** condition. This needs further analysis on the brightness-class level.

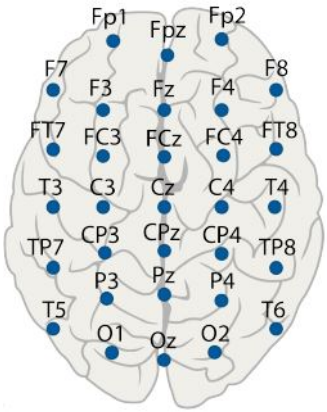
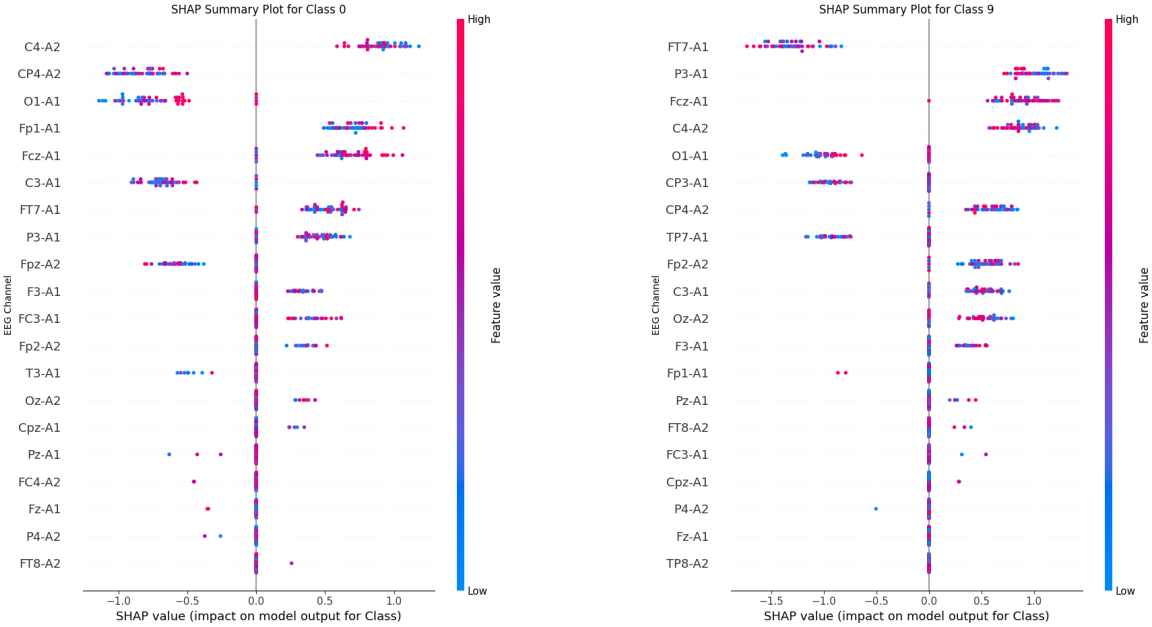


Figure 26: Brain map of the 31 EEG channels used in the experiments (Islam et al., 2022).

3.5.2 Channel-wise SHAP analysis for the lowest and highest brightness classes

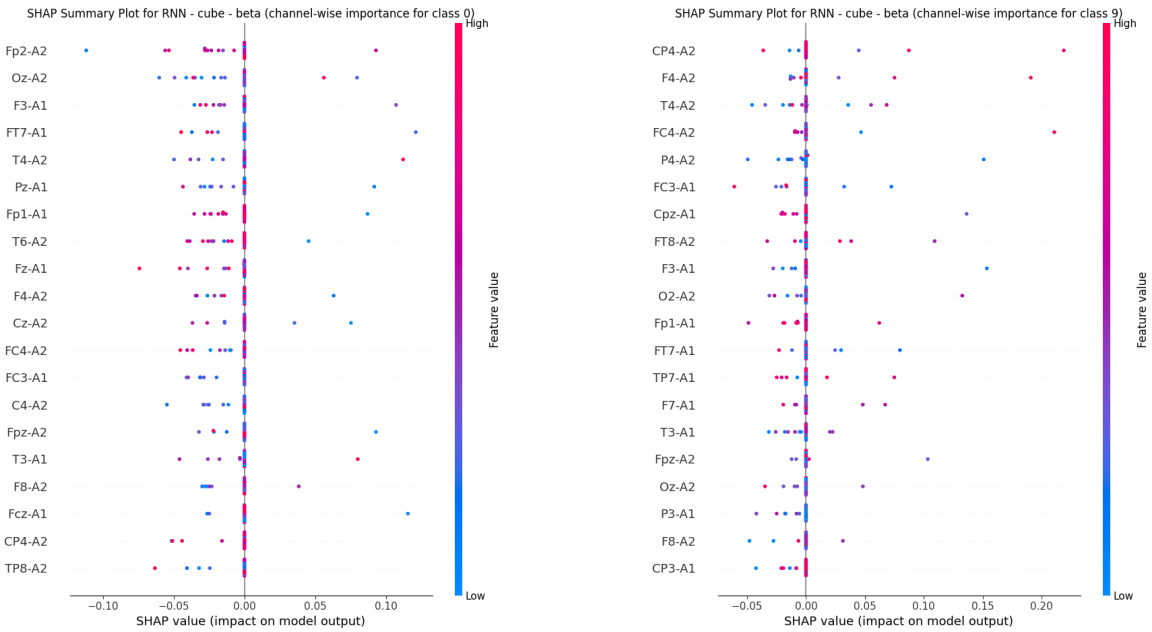
Cube-beta To understand how the models decide on brightness levels for the Necker cube stimulus using beta band activity, I looked at channel-wise SHAP summary plots. This analysis focused on Class 0 (lowest brightness) and Class 9 (highest brightness) for the **cube-beta** condition, building on the insights from the previously-discussed overall channel importance bar plots. The beta frequency band is usually linked to higher-level cognitive processes like attention and maintaining a particular mental state or perception (refer to this section 2.1.1 for more about frequency bands significance), which is relevant for an

ambiguous figure like the Necker cube.



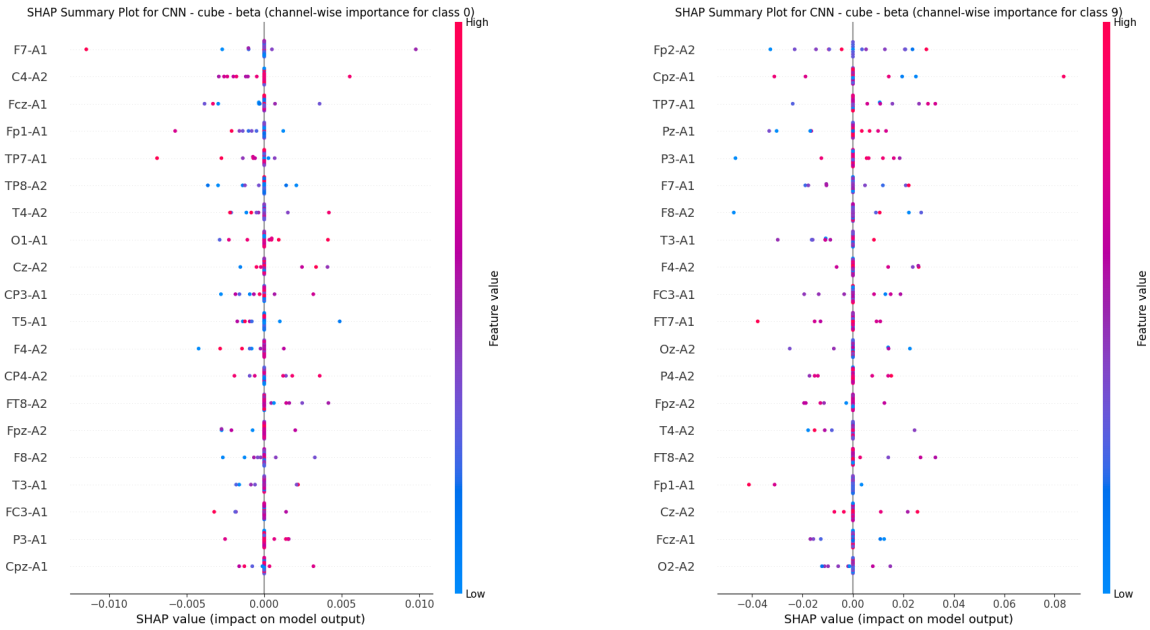
(a) 0.1 brightness (b) 1.0 brightness

Figure 27: Channel-wise SHAP summary for ANN-Cube-Beta.



(a) 0.1 brightness (b) 1.0 brightness

Figure 28: Channel-wise SHAP summary for RNN-Cube-Beta.



(a) 0.1 brightness

(b) 1.0 brightness

Figure 29: Channel-wise SHAP summary for CNN-Cube-Beta.

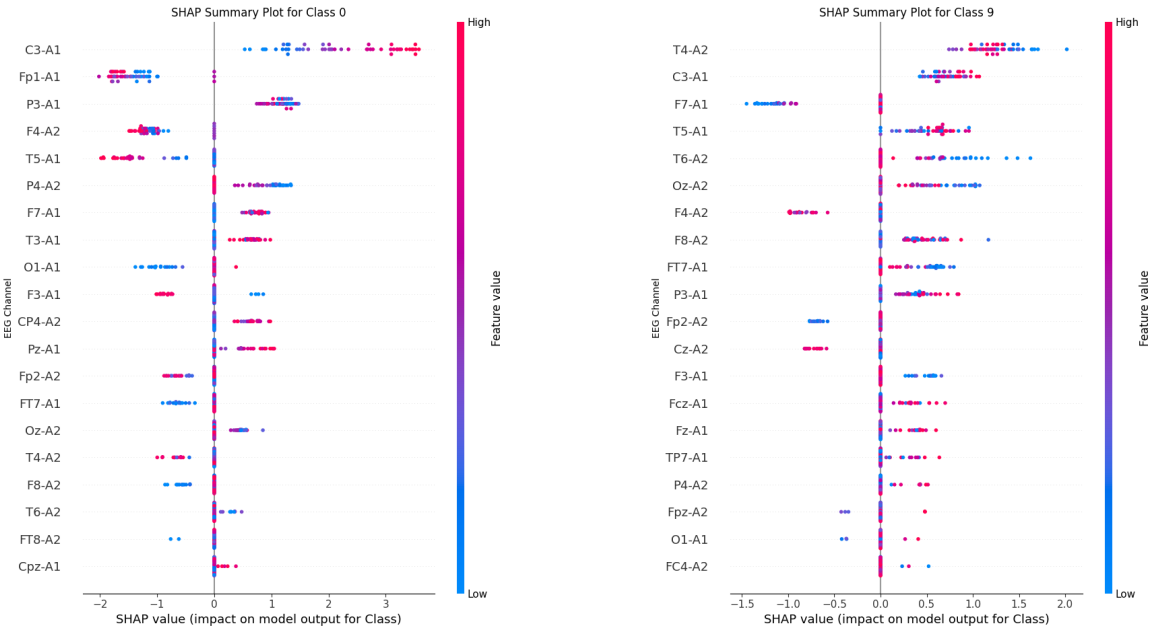
When predicting Class 9 (high brightness) for **cube-beta**, the RNN model relied clearly and significantly on increased beta band activity in several channels in the right-hemisphere, such as CP4-A2 (right centroparietal), F4-A2 (right frontal), T4-A2 (right temporal) and FC4-A2 (right frontocentral). Many of these right-sided channels (F4-A2, F8-A2, FC4-A2, Fpz-A2) were also among its top 10 overall important features, and these class-specific plots confirm that their high activity strongly pushes towards Class 9. This noticeable right-hemisphere beta engagement for a bright and clear Necker cube could indicate that the network has learned to associate a stable perception of the cube's orientation (which becomes clearer with brightness) with some specific activity patterns in the right side of the brain, reflecting continuous attention or a focused state of perception. The CNN-Transformer, for Class 9, highlighted high activity in Fp2-A2 (right prefrontal), a channel which also appeared in its overall top 10, but also in parietal channels like Pz-A1 and Cpz-A1 (located centrally, referenced to the left hemisphere). The CNN-Transformer's involved different areas than the RNN, right prefrontal and central-left parietal, suggesting a different way for processing the bright cube. The ANN model, for Class 9, primarily used increased activity in several left-hemisphere channels (like FT7-A1 and P3-A1), some of which were also in its overall top 10 list.

For Class 0 (low brightness) in the **cube-beta** condition, the obtained patterns were different as there was less lateralization (focus on one area of the brain). The ANN seemed to have connected lower activity in some of the left-hemisphere channels (such as O1 and

Fp1, both appearing in its overall top 10) with the class 0. The CNN-Transformer also showed contributions from left-hemisphere channels (high activity in F7-A1 and TP7-A1 for example, also in its overall top 10) for predicting low brightness, alongside some right-hemisphere activity (C4-A2, T4-A2). For the RNN predicting Class 0, its strategy was less one-sided compared to its strong right-hemisphere engagement for Class 9. While right-sided channels like Fp2-A2 and T4-A2 still contributed, other channels from its overall top 10 list (Oz-A2, T3-A1) showed varied impacts. This mixed beta pattern for the low-brightness ambiguous Necker cube could reflect the difficulty in forming a stable perception, leading to the involvement of attentional networks in both hemispheres as the brain tries to interpret the less clear stimulus.

To summarise, the channel-wise SHAP analysis for **cube-beta** shows model-dependent patterns for distinguishing brightness classes. The RNN consistently linked increased right-hemisphere beta activity (from channels also in its overall top 10 influential channels) with high brightness, potentially reflecting the way the brain perceives the clear cube, and aligning with literature suggesting right hemisphere dominance for spatial recognition tasks (Witelson, 1976). For low brightness, the patterns became more mixed across all models and less lateralized. This could be understood as follows: the processing of a more ambiguous stimuli where a clear and dominant perceptual interpretation is hard to establish, leads to more distributed beta band engagement across both hemispheres. It is important to remind here that the model's task is not shape recognition, but brightness class prediction, and therefore unrecognised patterns by visual perception-related literature can be expected.

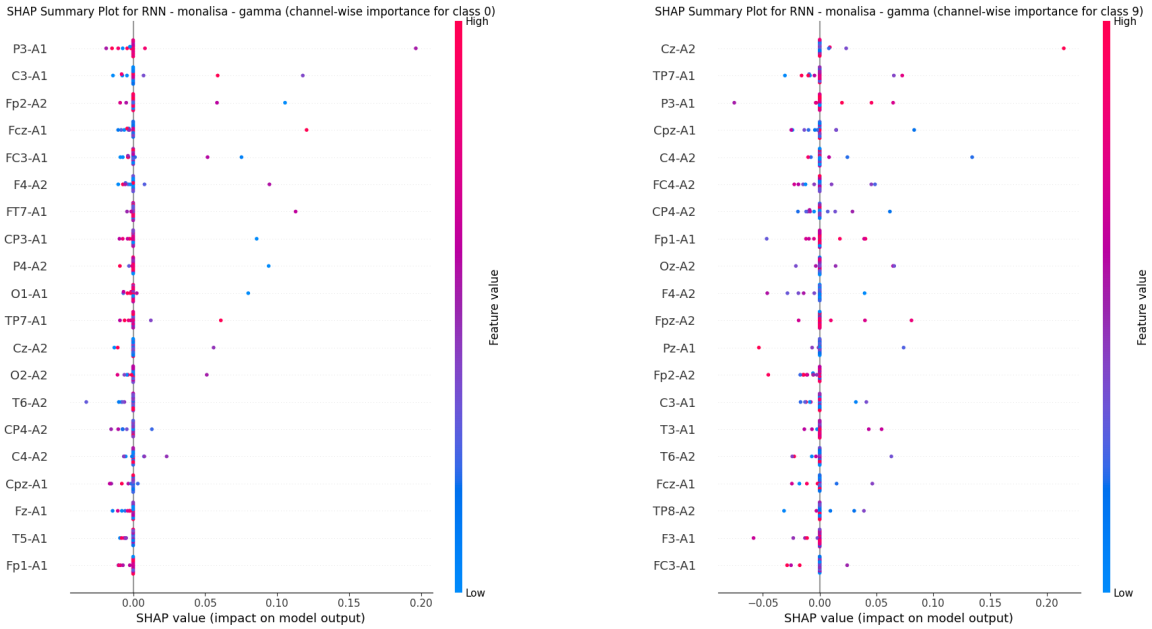
Monalisa-gamma Next, I examined the channel-wise SHAP summary plots for the **monalisa-gamma** condition, also focusing on how the studied models predict Class 0 (lowest brightness) and Class 9 (highest brightness) for the complex Mona Lisa image using gamma band activity. This band is often linked to detailed visual processing and sensory information integration (or a state of active learning); and the Mona Lisa portrait is a complex and intriguing work of art that often evokes a high level of psychological engagement in individuals, hence the interest in this condition. The following SHAP plots will showcase in more details how the activity level in each EEG channel influences the prediction for these specific brightness classes.



(a) 0.1 brightness

(b) 1.0 brightness

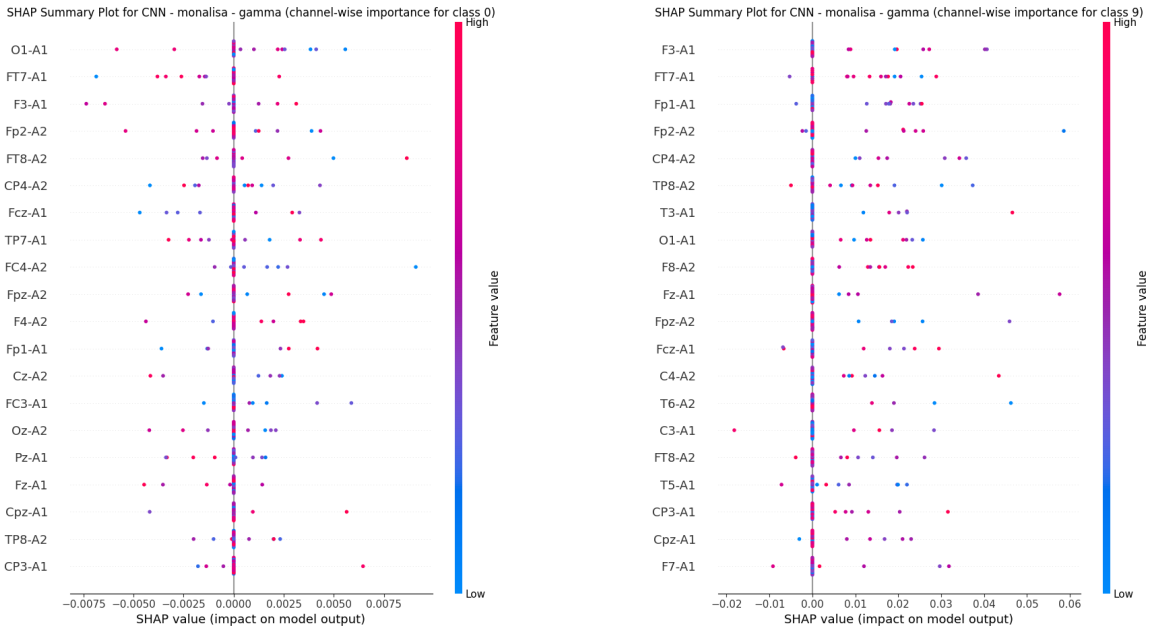
Figure 30: Channel-wise SHAP summary for ANN-Monalisa-Gamma.



(a) 0.1 brightness

(b) 1.0 brightness

Figure 31: Channel-wise SHAP summary for RNN-Monalisa-Gamma.



(a) 0.1 brightness

(b) 1.0 brightness

Figure 32: Channel-wise SHAP summary for CNN-Monalisa-Gamma.

When predicting Class 9 (high brightness) for the **monalisa-gamma** stimulus, the RNN model showed that high gamma power in the channel Cz-A2 (located near the top-center of the head, with a right-hemisphere reference) was a strong driver towards a bright-class prediction; Cz-A2 was also among its top overall important channels. This activity associated with Cz-A2 was supported by increased gamma in channels across both hemispheres, including right-hemisphere channels like C4-A2, CP4-A2, FC4-A2 and left-hemisphere channels such as P3-A1, TP7-A1, C3-A1. This bilateral increase in gamma activity, with a significant contribution from Cz-A2, may reflect how the brain processes intense (in terms of brightness and complexity of details) visual information, being in a state of heightened engagement. Frontal channels in both hemispheres like Fp1-A1 and Fp2-A2 also contributed positively with high gamma, consistent with increased alertness. The CNN-Transformer, for Class 9, also highlighted channels in the left frontal region (F3-A1, FT7-A1, Fp1-A1) and the right prefrontal region (Fp2-A2, CP4-A2, TP8-A2), many of which were in its overall top 10 features, showing a focus on frontal brain activity for identifying high brightness. The ANN model, for Class 9, showed that high gamma activity in channels such as T4-A2 (right temporal), C3-A1 (left central), and F7-A1 (left frontal) strongly contributed to predicting high brightness.

For predicting Class 0 (low brightness) with the **monalisa-gamma** stimulus, the RNN model mainly relied on decreased gamma activity. This pattern was clearly evident in several left-hemisphere channels (parietal P3-A1, central C3-A1, temporofrontal FT7-A1 and CP3-A1), where low gamma power strongly pushed the prediction towards the low-brightness

class. Some right-hemisphere frontal channels, such as Fp2-A2 and F4-A2, also showed that low gamma activity contributed to this classification. While this pattern of low gamma power leading to the low-brightness class was mainly observed in the left hemisphere, the involvement of these right channels suggests, to a certain degree, a bilateral aspect to this pattern. Although, the plots show that the right-hemisphere had less influence on this particular class. This suggests that low brightness scenes might not only cause weaker gamma signals generally but could specifically lead to reduced gamma in networks associated with attention and visual exploration, with a notable impact observed in left-hemisphere regions and some right frontal areas, maybe because there are less details to scan in a dark image. The CNN-Transformer for this class also indicated that low gamma activity in left frontal regions (such as F3-A1) contributed to the prediction, alongside increased gamma in O1-A1 and FT7-A1 channels. The ANN model indicated that lower gamma activity in the left prefrontal channel Fp1-A1 and the left frontal channel F7-A1 contributed to predicting Class 0

The SHAP analysis for **monalisa-gamma** suggests the following in summary; the RNN model distinguishes brightness levels using a distributed pattern of gamma activity: strong, widespread gamma activity (with significant input from Cz-A2 and other channels) for high brightness, likely indicating an intense visual input; and targeted gamma suppression, especially in left-hemisphere networks, for low brightness. This indicates that for classifying the brightness of this complex image, the model relies on interpreting gamma activity patterns distributed on both hemisphere, with no clear dominance of one over the other, rather than dominantly on specific known face-processing regions from the right hemisphere as literature suggests when it comes to facial recognition (Benton, 1980; Levine et al., 1988; Ishai et al., 2005). In other words, the model does not seem to generally rely on known facial-recognition related channels to achieve its task, that is brightness class prediction. The CNN-Transformer and ANN also showed class-specific strategies, often involving their previously identified top overall channels.

4 Conclusion

This study investigated the application and evaluation of different deep learning architectures for analysing brain activity time series data in the context of visual perception, focusing on classifying 10 brightness levels, ranging from 0.1 to 1. The previously-made experiments consisted of recording electroencephalography (EEG) data obtained from 5 participants being exposed to two distinct visual stimuli, the **Necker cube** and the **Mona Lisa** portrait,

through different trials involving different degrees of brightness, across 31 spatial channels or electrodes. The raw EEG data was separated into five standard frequency bands (delta δ, theta θ, alpha α, beta β and gamma γ) with a 4th order Butterworth filter, each band representing a separate dataset for a given visual stimuli. This research built on the previous work of Islam et al. (2022) by introducing different DL models, and by refining the data processing pipeline in order to make it more relevant to real-world applications such as Brain-Computer Interfaces (BCIs).

The investigation involved the implementation and comparative analysis of three principal model architectures: a standard Artificial Neural Network (ANN) serving as a baseline, a novel hybrid Recurrent Neural Network (RNN) architecture augmented with a convolutional front-end and multi-head self-attention mechanisms, and a custom Convolutional Neural Network (CNN)-Transformer model based on the EEGNet architecture and the transformer model. A key change in my methodology involved processing the continuous EEG signals in smaller, fixed-length temporal sequences of length 100 (0.4 seconds worth of data), presenting a more challenging but practically applicable classification task compared to using full-trial recordings. The evaluation step relied not only on quantitative performance metrics (accuracy, F1, precision, recall and training time) across the different EEG frequency bands and visual stimuli datasets, but also on qualitative analysis of learning dynamics and classification patterns reflected in confusion matrices. In addition, the study integrated Explainable AI (XAI) techniques, SHAP (SHapley Additive exPlanation) specifically, to gain more insight into the contribution of different EEG channels, to therefore identify which were most influential in driving the models' predictions for different brightness classes.

The quantitative results presented in the preceding sections demonstrated clear differences in the abilities of the evaluated architectures regarding classification accuracy, generalisation to unseen data, and computational efficiency. The learning dynamics observed during training highlighted how overfitting can affect the models performances, the baseline ANN in particular being the most susceptible to this problem. Conversely, the DL architectures designed to process temporal sequences (the hybrid RNN and CNN-Transformer) demonstrated improved generalisation and higher performance levels. Among them, the hybrid RNN consistently demonstrated superior overall effectiveness and efficiency across the investigated conditions, showcasing the benefits of incorporating explicit temporal dependencies in the models and its practical advantages in terms of accuracy and training time.

Furthermore, applying SHAP provided crucial interpretability by revealing how the models used the EEG signals for classification. This analysis showed that each model

learned to prioritize distinct brain regions and specific patterns of activity within those regions to predict brightness levels. For instance, the hybrid RNN frequently attributed high importance to right-hemisphere channels, particularly for classifying high brightness in the beta band, a pattern that can be related to theories of visual processing lateralization. However, other models highlighted different brain areas, suggesting varied learned strategies. This interpretability aspect is vital for building trust in ML models applied to sensitive domains like neuroscience and healthcare.

5 Limitations and future work

This research highlighted the importance of carefully selecting and designing DL architectures that are suitable to the characteristics of the spatio-temporal EEG data, especially when aiming for both high performance and interpretability to meet the requirements of some real-world applications such as BCIs. However, I have faced a number of obstacles that constrained the scope of this work. For instance, the available EEG data was recorded for only five individuals with 2 visual stimuli, thus clearly limiting model generalisability. Also, there is the hardware limitations that comes with the free tier subscription of the T4 GPU computer in Google Colab. This imposed some unwanted but required simplifications, such as the reduced NAS search space, or the use of truncated sequences for the SHAP values calculations.

Despite the previous restrictions, the results obtained were quite insightful and promising. These findings could serve as a foundation to build better and clearer explainable Machine Learning models for complex brain signal analysis. Future work could extend this investigation by exploring alternative data representations, refining the hybrid architectures, adjusting the sequence length (depending on research goals), and possibly applying these models on datasets with a wider range of cognitive tasks and involving more individuals, for better generalisability.

Disclaimer

There is minor use of generative AI in this work, strictly for information gathering, language polishing and grammatical correction, in order to improve clarity. AI was not used to generate any of the content itself. This is my original work and I take full responsibility for it.
Some prompt examples:
Is this passage grammatically correct?

(passage)

Does this sentence need further punctuations?

(sentence)

Explain the following passage in simpler terms:

(passage)

Organise the following results into a table in latex code:

(Python dictionary)

References

Abdullah, T. A., Zahid, M. S. M., and Ali, W. (2021). A review of interpretable ml in healthcare: taxonomy, applications, challenges, and future directions. *Symmetry*, 13(12):2439.

Arutiunian, V., Arcara, G., Buyanova, I., Buivolova, O., Davydova, E., Pereverzeva, D., Sorokin, A., Tyushkevich, S., Mamokhina, U., Danilina, K., et al. (2023). Event-related desynchronization of meg alpha-band oscillations during simultaneous presentation of audio and visual stimuli in children with autism spectrum disorder. *Brain Sciences*, 13(9):1313.

Belle, V. and Papantonis, I. (2021). Principles and practice of explainable machine learning. *Frontiers in big Data*, 4:688969.

Benton, A. L. (1980). The neuropsychology of facial recognition. *American Psychologist*, 35(2):176.

Bi, W. L., Hosny, A., Schabath, M. B., Giger, M. L., Birkbak, N. J., Mehrtash, A., Allison, T., Arnaout, O., Abbosh, C., Dunn, I. F., et al. (2019). Artificial intelligence in cancer imaging: clinical challenges and applications. *CA: a cancer journal for clinicians*, 69(2):127–157.

Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.

Corballis, P. M. (2003). Visuospatial processing and the right-hemisphere interpreter. *Brain and cognition*, 53(2):171–176.

Craik, A., He, Y., and Contreras-Vidal, J. L. (2019). Deep learning for electroencephalogram (eeg) classification tasks: a review. *Journal of neural engineering*, 16(3):031001.

Elsken, T., Metzen, J. H., and Hutter, F. (2019). Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.

Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. (2016). *Deep learning*, volume 1. MIT press Cambridge.

Graves, A. (2013). Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*.

Gui, X., Chuansheng, C., Zhong-Lin, L., and Qi, D. (2010). Brain imaging techniques and their applications in decision-making research. *Xin li xue bao. Acta psychologica Sinica*, 42(1):120.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*,

9(8):1735–1780.

Hussain, I. and Park, S. J. (2020). Healthsos: Real-time health monitoring system for stroke prognostics. *IEEE Access*, 8:213574–213586.

Ishai, A., Schmidt, C. F., and Boesiger, P. (2005). Face perception is mediated by a distributed cortical network. *Brain research bulletin*, 67(1-2):87–93.

Islam, R., Andreev, A. V., Shusharina, N. N., and Hramov, A. E. (2022). Explainable machine learning methods for classification of brain states during visual perception. *Mathematics*, 10(15):2819.

Karim, F., Majumdar, S., Darabi, H., and Chen, S. (2017). Lstm fully convolutional networks for time series classification. *IEEE access*, 6:1662–1669.

Kumar, J. S. and Bhuvaneswari, P. (2012). Analysis of electroencephalography (eeg) signals and its categorization–a study. *Procedia engineering*, 38:2525–2536.

Lawhern, V. J., Solon, A. J., Waytowich, N. R., Gordon, S. M., Hung, C. P., and Lance, B. J. (2018). Eegnet: a compact convolutional neural network for eeg-based brain–computer interfaces. *Journal of neural engineering*, 15(5):056013.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.

Levine, S. C., Banich, M. T., and Koch-Weser, M. P. (1988). Face recognition: a general or specific right hemisphere capacity? *Brain and cognition*, 8(3):303–325.

Luck, S. J. (2014). *An introduction to the event-related potential technique*. MIT press.

Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30.

Makeig, S., Bell, A., Jung, T.-P., and Sejnowski, T. J. (1995). Independent component analysis of electroencephalographic data. *Advances in neural information processing systems*, 8.

McFarland, D. J. and Wolpaw, J. R. (2017). Eeg-based brain–computer interfaces. *current opinion in Biomedical Engineering*, 4:194–200.

Mosher, J. C., Leahy, R. M., and Lewis, P. S. (1999). Eeg and meg: forward solutions for inverse methods. *IEEE Transactions on biomedical engineering*, 46(3):245–259.

Musk, E. et al. (2019). An integrated brain-machine interface platform with thousands of channels. *Journal of medical Internet research*, 21(10):e16194.

Pascanu, R., Mikolov, T., and Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. Pmlr.

Pisarchik, A. N., Maksimenko, V. A., Andreev, A. V., Frolov, N. S., Makarov, V. V., Zhuravlev, M. O., Runnova, A. E., and Hramov, A. E. (2019). Coherent resonance in the distributed cortical network during sensory information processing. *Scientific Reports*, 9(1):18325.

Robiul, I., Alexander E., H., and Andrey, A. (2022). Eeg data set. Figshare. 10.6084/m9.figshare.19849009.v2.

Roscher, R., Bohn, B., Duarte, M. F., and Garcke, J. (2020). Explainable machine learning for scientific insights and discoveries. *Ieee Access*, 8:42200–42216.

Roser, M. E., Fiser, J., Aslin, R. N., and Gazzaniga, M. S. (2011). Right hemisphere dominance in visual statistical learning. *Journal of cognitive neuroscience*, 23(5):1088–1099.

Rozemberczki, B., Watson, L., Bayer, P., Yang, H.-T., Kiss, O., Nilsson, S., and Sarkar, R. (2022). The shapley value in machine learning. *arXiv preprint arXiv:2202.05594*.

Schirrmeister, R. T., Springenberg, J. T., Fiederer, L. D. J., Glasstetter, M., Eggensperger, K., Tangermann, M., Hutter, F., Burgard, W., and Ball, T. (2017). Deep learning with convolutional neural networks for eeg decoding and visualization. *Human brain mapping*, 38(11):5391–5420.

Shapley, L. S. et al. (1953). A value for n-person games.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

Witelson, S. F. (1976). Sex and the single hemisphere: Specialization of the right hemisphere for spatial processing. *Science*, 193(4251):425–427.

Xu, G., Shen, X., Chen, S., Zong, Y., Zhang, C., Yue, H., Liu, M., Chen, F., and Che, W. (2019). A deep transfer convolutional neural network framework for eeg signal classification. *IEEE Access*, 7:112767–112776.

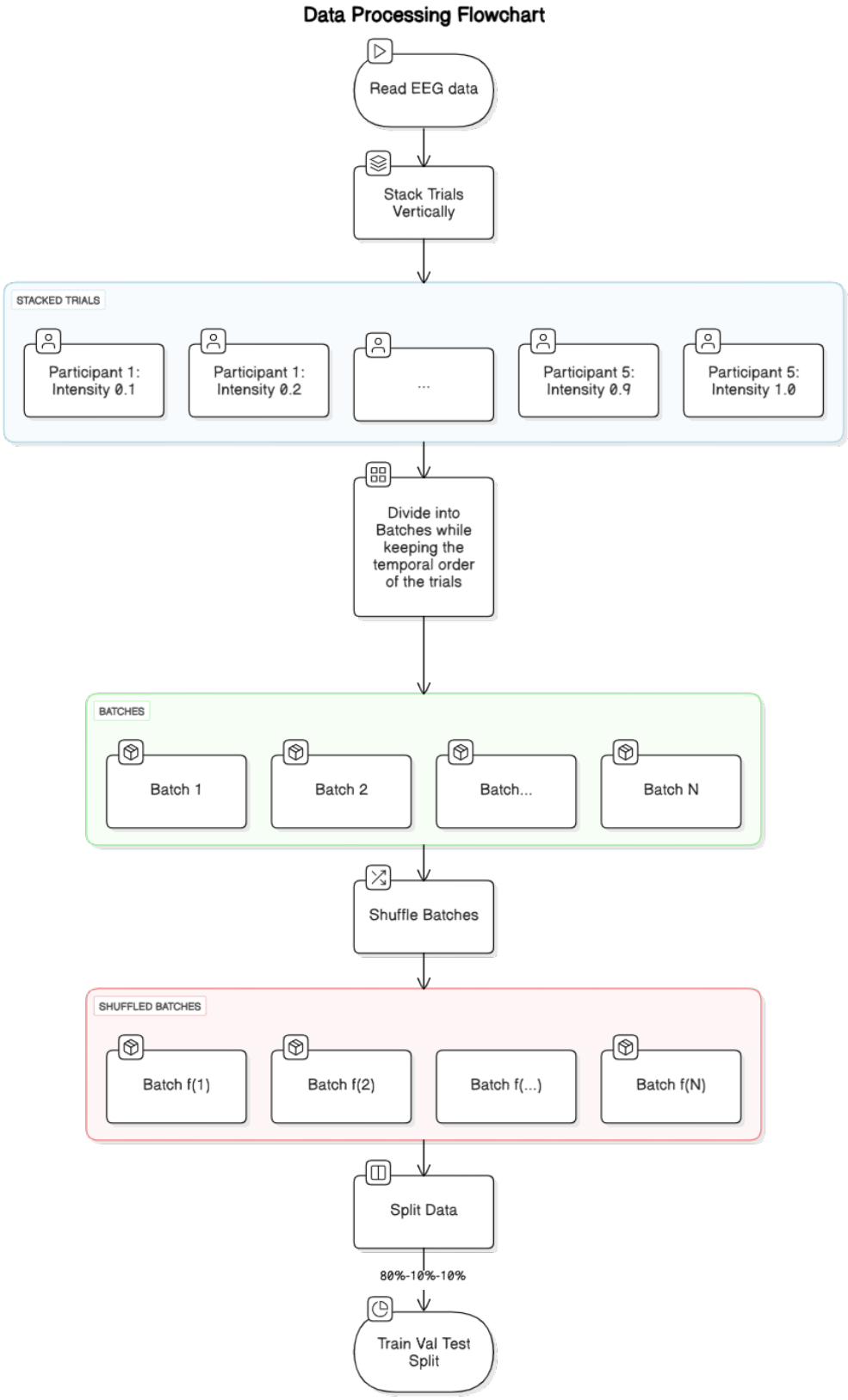
Ying, X. (2019). An overview of overfitting and its solutions. In *Journal of physics: Conference series*, volume 1168, page 022022. IOP Publishing.

Yurchenko, A., Arutiunian, V., Shitova, N. M., Bergelson, M., and Dragoy, O. (2023). Register switching involving lexical-semantic processing in russian: An erp study. *Journal of Neurolinguistics*, 65:101111.

Zhou, M., Tian, C., Cao, R., Wang, B., Niu, Y., Hu, T., Guo, H., and Xiang, J. (2018). Epileptic seizure detection based on eeg signals and cnn. *Frontiers in neuroinformatics*, 12:95.

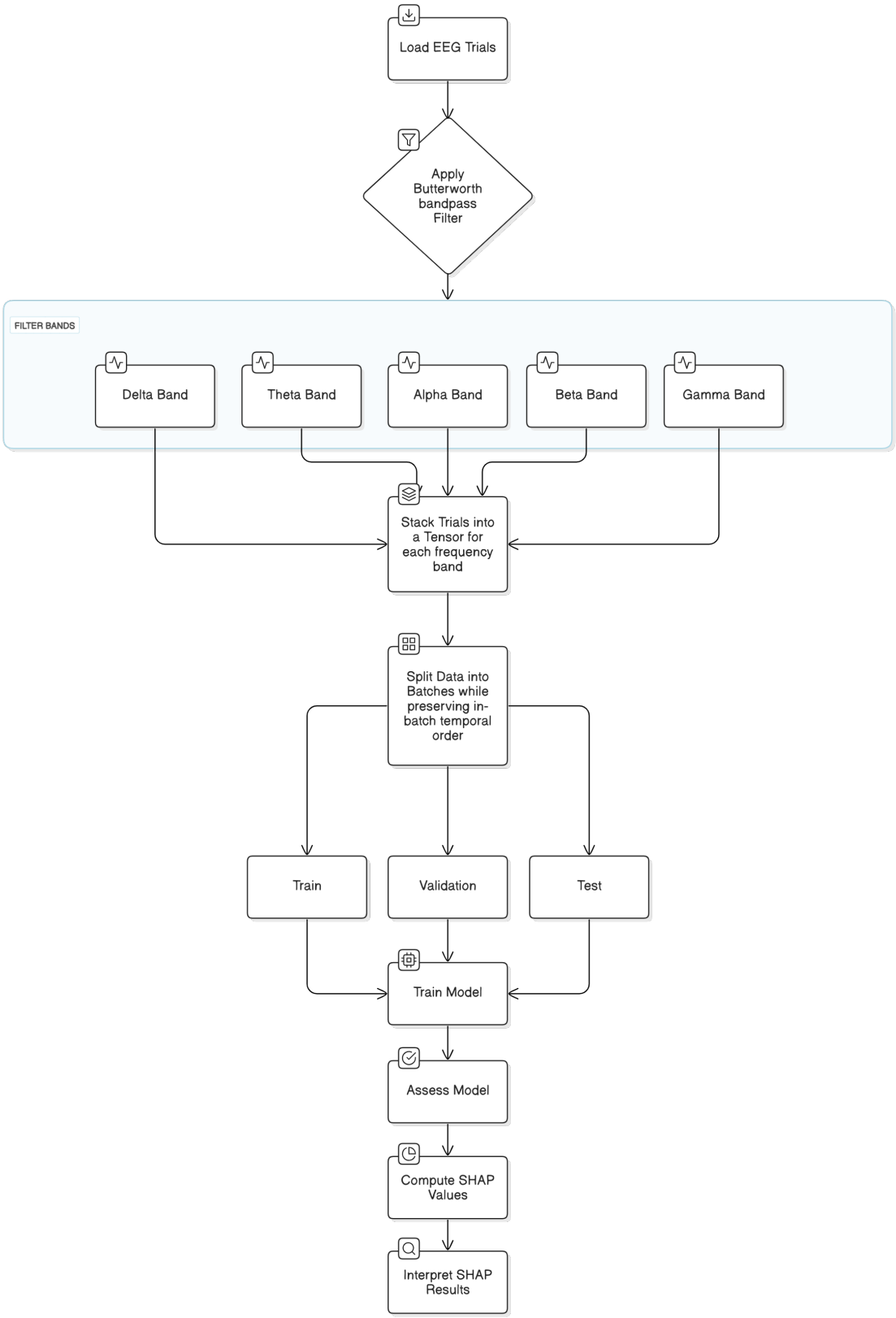
A Appendix

A.1 Data split for training



A.2 Analysis process

EEG Data Analysis with interpretable Machine Learning Flow chart



A.3 RNN-based model NAS results for alpha band of the cube dataset


```
{
  "{'h_size_1': 128, 'h_size_2': 256, 'h_size_3': 128}": {
    "accuracy": 61.23,
    "f1": 61.07,
    "recall": 61.47,
    "precision": 61.27,
    "execution_time": 264.30,
    "score": 53.90
  },
  "{'h_size_1': 256, 'h_size_2': 256, 'h_size_3': 128}": {
    "accuracy": 62.00,
    "f1": 62.00,
    "recall": 62.29,
    "precision": 62.31,
    "execution_time": 328.60,
    "score": 52.99
  },
  "{'h_size_1': 128, 'h_size_2': 128, 'h_size_3': 128}": {
    "accuracy": 55.20,
    "f1": 55.00,
    "recall": 55.28,
    "precision": 55.79,
    "execution_time": 178.40,
    "score": 50.32
  },
  "{'h_size_1': 256, 'h_size_2': 128, 'h_size_3': 128}": {
    "accuracy": 57.25,
    "f1": 57.12,
    "recall": 57.25,
    "precision": 57.47,
    "execution_time": 251.50,
    "score": 50.27
```

```
},
"{'h_size_1': 128, 'h_size_2': 256, 'h_size_3': 256}": {
  "accuracy": 60.08,
  "f1": 60.28,
  "recall": 60.26,
  "precision": 60.61,
  "execution_time": 393.30,
  "score": 49.36
},
"{'h_size_1': 128, 'h_size_2': 128, 'h_size_3': 256}": {
  "accuracy": 57.13,
  "f1": 57.00,
  "recall": 57.37,
  "precision": 57.36,
  "execution_time": 297.40,
  "score": 48.92
},
"{'h_size_1': 128, 'h_size_2': 512, 'h_size_3': 256}": {
  "accuracy": 64.57,
  "f1": 64.46,
  "recall": 64.72,
  "precision": 64.74,
  "execution_time": 599.40,
  "score": 47.95
},
"{'h_size_1': 256, 'h_size_2': 128, 'h_size_3': 256}": {
  "accuracy": 57.90,
  "f1": 58.16,
  "recall": 58.25,
  "precision": 58.57,
  "execution_time": 371.30,
  "score": 47.87
},
"{'h_size_1': 128, 'h_size_2': 512, 'h_size_3': 128}": {
```

```
    "accuracy": 59.82,
    "f1": 59.71,
    "recall": 60.12,
    "precision": 59.65,
    "execution_time": 460.90,
    "score": 47.01
  },
  "{'h_size_1': 512, 'h_size_2': 256, 'h_size_3': 128}": {
    "accuracy": 62.90,
    "f1": 62.84,
    "recall": 63.05,
    "precision": 63.04,
    "execution_time": 580.20,
    "score": 46.83
  },
  "{'h_size_1': 256, 'h_size_2': 512, 'h_size_3': 128}": {
    "accuracy": 62.00,
    "f1": 61.94,
    "recall": 61.80,
    "precision": 62.67,
    "execution_time": 564.80,
    "score": 46.39
  },
  "{'h_size_1': 256, 'h_size_2': 256, 'h_size_3': 256}": {
    "accuracy": 58.92,
    "f1": 58.96,
    "recall": 59.39,
    "precision": 59.68,
    "execution_time": 464.00,
    "score": 46.29
  },
  "{'h_size_1': 512, 'h_size_2': 128, 'h_size_3': 256}": {
    "accuracy": 61.49,
    "f1": 61.23,
```

```
    "recall": 61.70,
    "precision": 61.30,
    "execution_time": 565.80,
    "score": 45.70
  },
  "{'h_size_1': 512, 'h_size_2': 128, 'h_size_3': 128}": {
    "accuracy": 58.28,
    "f1": 58.13,
    "recall": 58.29,
    "precision": 58.15,
    "execution_time": 453.90,
    "score": 45.60
  },
  "{'h_size_1': 512, 'h_size_2': 256, 'h_size_3': 256}": {
    "accuracy": 60.85,
    "f1": 60.90,
    "recall": 61.03,
    "precision": 60.87,
    "execution_time": 697.20,
    "score": 41.54
  },
  "{'h_size_1': 256, 'h_size_2': 256, 'h_size_3': 512}": {
    "accuracy": 63.93,
    "f1": 63.55,
    "recall": 63.72,
    "precision": 63.90,
    "execution_time": 821.60,
    "score": 40.95
  },
  "{'h_size_1': 256, 'h_size_2': 128, 'h_size_3': 512}": {
    "accuracy": 59.95,
    "f1": 59.96,
    "recall": 60.40,
    "precision": 60.30,
```

```
    "execution_time": 701.00,
    "score": 40.64
  },
  "{'h_size_1': 256, 'h_size_2': 512, 'h_size_3': 256}": {
    "accuracy": 60.46,
    "f1": 60.32,
    "recall": 60.63,
    "precision": 60.65,
    "execution_time": 717.60,
    "score": 40.56
  },
  "{'h_size_1': 512, 'h_size_2': 512, 'h_size_3': 128}": {
    "accuracy": 62.64,
    "f1": 62.38,
    "recall": 62.70,
    "precision": 62.60,
    "execution_time": 796.51,
    "score": 40.44
  },
  "{'h_size_1': 128, 'h_size_2': 256, 'h_size_3': 512}": {
    "accuracy": 60.72,
    "f1": 60.42,
    "recall": 60.55,
    "precision": 60.95,
    "execution_time": 736.00,
    "score": 40.20
  },
  "{'h_size_1': 128, 'h_size_2': 128, 'h_size_3': 512}": {
    "accuracy": 56.87,
    "f1": 56.76,
    "recall": 56.91,
    "precision": 56.90,
    "execution_time": 615.80,
    "score": 39.74
```

```
},
"{'h_size_1': 512, 'h_size_2': 256, 'h_size_3': 512}": {
  "accuracy": 62.64,
  "f1": 62.38,
  "recall": 62.70,
  "precision": 62.60,
  "execution_time": 986.20,
  "score": 38.55
},
"{'h_size_1': 512, 'h_size_2': 512, 'h_size_3': 256}": {
  "accuracy": 65.34,
  "f1": 65.01,
  "recall": 65.17,
  "precision": 65.05,
  "execution_time": 957.60,
  "score": 38.55
},
"{'h_size_1': 512, 'h_size_2': 128, 'h_size_3': 512}": {
  "accuracy": 55.07,
  "f1": 55.19,
  "recall": 55.06,
  "precision": 55.67,
  "execution_time": 875.10,
  "score": 30.91
},
"{'h_size_1': 256, 'h_size_2': 512, 'h_size_3': 512}": {
  "accuracy": 60.21,
  "f1": 60.19,
  "recall": 60.18,
  "precision": 60.60,
  "execution_time": 1134.10,
  "score": 28.77
},
"{'h_size_1': 512, 'h_size_2': 512, 'h_size_3': 512}": {
```

```
    "accuracy": 59.18,
    "f1": 58.90,
    "recall": 59.10,
    "precision": 59.81,
    "execution_time": 1342.20,
    "score": 21.92
  }
}
```


A.4 Code

For the full source code of this research, please refer to my GitHub:

https://github.com/MaruwanDono/EEG_DeepLearning_Classification.git

Figure 33: GitHub repository.

The main models code:

A.4.1 ANN

```
class EEG_ANN(nn.Module):
    def __init__(self, input_shape, n_classes):
        super(EEG_ANN, self).__init__()
        batch_size, num_channels = input_shape

        # Hidden layers
        self.fc1 = nn.Linear(num_channels, 2500) # Input to first hidden layer
        self.fc2 = nn.Linear(2500, 1000)         # Second hidden layer
```

```
        self.fc3 = nn.Linear(1000, 500)          # Third hidden layer
        self.fc4 = nn.Linear(500, 200)           # Fourth hidden layer
        self.fc5 = nn.Linear(200, 100)           # Fifth hidden layer
        self.fc6 = nn.Linear(100, 50)            # Sixth hidden layer
        self.fc7 = nn.Linear(50, 25)             # Seventh hidden layer
        self.fc8 = nn.Linear(25, 15)             # Eighth hidden layer

        # Output layer
        self.fc_out = nn.Linear(15, n_classes)   # Output layer

        # Activation functions
        self.tanh = nn.Tanh()                  # Tanh activation
        self.softmax = nn.Softmax(dim=1)       # Softmax activation

    def forward(self, x):
        # Pass through hidden layers with tanh activation
        x = self.tanh(self.fc1(x))
        x = self.tanh(self.fc2(x))
        x = self.tanh(self.fc3(x))
        x = self.tanh(self.fc4(x))
        x = self.tanh(self.fc5(x))
        x = self.tanh(self.fc6(x))
        x = self.tanh(self.fc7(x))
        x = self.tanh(self.fc8(x))

        # Output layer with softmax activation
        x = self.fc_out(x)

        return x
```

A.4.2 Hybrid RNN

```
class MultiHeadAttention(nn.Module):
    def __init__(self, embed_dim, num_heads=1, dropout=0.3):
```

```
        super(MultiHeadAttention, self).__init__()
        assert embed_dim % num_heads == 0, \
        "embed_dim must be divisible by num_heads"
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.dim_per_head = embed_dim // num_heads

        # Linear projections for query, key, and value.
        self.q_linear = nn.Linear(embed_dim, embed_dim)
        self.k_linear = nn.Linear(embed_dim, embed_dim)
        self.v_linear = nn.Linear(embed_dim, embed_dim)

        # Final linear projection after combining heads.
        self.out_linear = nn.Linear(embed_dim, embed_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        """
        Args:
            x: input tensor of shape (batch_size, seq_length, embed_dim)
        Returns:
            A tensor of shape (batch_size, seq_length, embed_dim) after
            applying multi-head self-attention.
        """
        B, T, _ = x.shape

        # Linear projections
        Q = self.q_linear(x)  # (B, T, embed_dim)
        K = self.k_linear(x)  # (B, T, embed_dim)
        V = self.v_linear(x)  # (B, T, embed_dim)

        # Reshape for multiple heads: (B, T, num_heads, dim_per_head)
        # and then transpose to shape (B, num_heads, T, dim_per_head)
        Q = Q.view(B, T, self.num_heads, self.dim_per_head).transpose(1, 2)
```

```
        K = K.view(B, T, self.num_heads, self.dim_per_head).transpose(1, 2)
        V = V.view(B, T, self.num_heads, self.dim_per_head).transpose(1, 2)

        # Scaled dot-product attention.
        scores = torch.matmul(Q, K.transpose(-2, -1)) / self.dim_per_head**0.5
        attn_weights = F.softmax(scores, dim=-1) #(B, num_heads, T, T)
        attn_output = torch.matmul(attn_weights, V) #(B, num_heads, T, dim_per_head)

        # Combine heads: transpose and reshape to (B, T, embed_dim)
        attn_output = attn_output.transpose(1, 2).contiguous().view(
            B, T, self.embed_dim
        )

        # Final linear projection and dropout.
        attn_output = self.out_linear(attn_output)
        attn_output = self.dropout(attn_output)

        return attn_output


class EEG_RNN(nn.Module):
    def __init__(self,
                input_shape,
                n_classes,
                rnn_layers = (128, 256, 128),
                bidirectional=True):
        super(EEG_RNN, self).__init__()
        batch_size, sequence_length, num_channels = input_shape
        h_conv1 = 128
        h_size_1, h_size_2, h_size_3 = rnn_layers
        dirs = (2 if bidirectional else 1)

        # CNN front layer
        self.conv1    = nn.Conv1d(
```

```
                    in_channels=num_channels,
                    out_channels=h_conv1,
                    kernel_size=3,
                    padding=1
                    )
self.bn_conv  = nn.BatchNorm1d(h_conv1)
self.dropout0 = nn.Dropout(0.2)

# Hidden layers
#RNN 1
self.h1 = nn.LSTM(input_size=h_conv1,
                  hidden_size=h_size_1,
                  num_layers=1,
                  batch_first=True,
                  bidirectional=bidirectional) # First layer
self.ln1 = nn.LayerNorm(h_size_1*dirs)
self.dropout1 = nn.Dropout(0.3)
# RNN 2
self.h2 = nn.LSTM(input_size=h_size_1*dirs,
                  hidden_size=h_size_2,
                  num_layers=1,
                  batch_first=True,
                  bidirectional=bidirectional) # 2nd  layer
self.ln2 = nn.LayerNorm(h_size_2*dirs)
self.dropout2 = nn.Dropout(0.3)
# RNN 3
self.h3 = nn.LSTM(input_size=h_size_2*dirs,
                  hidden_size=h_size_3,
                  num_layers=1,
                  batch_first=True,
                  bidirectional=bidirectional) # 3rd layer
self.ln3 = nn.LayerNorm(h_size_3*dirs)
self.dropout3 = nn.Dropout(0.3)
```

```
        # Attention
        self.attn = nn.Linear(h_size_3*dirs, 1)
        self.attn_multi = MultiHeadAttention(
                            embed_dim=h_size_3*dirs,
                            num_heads=16,
                            dropout=0.2
                            )

        # Output fc layer(s)
        self.fc_out = nn.Linear(h_size_3*dirs, n_classes)

        # Activation functions
        self.tanh = nn.Tanh()
        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)


    def forward(self, x):
        # x: (batch_size, sequence_length, n_channels)
        # Initial CNN layer
        x = x.permute(0, 2, 1) # -> (batch, n_channels, seq_len)
        x = self.conv1(x)      # -> (batch, h_conv1, seq_len)
        x = self.bn_conv(x)
        x = self.tanh(x)
        x = self.dropout0(x)
        x = x.permute(0, 2, 1)

        # Pass through hidden layers with tanh activation
        output, _ = self.h1(x) # -> (batch_size, seq_len, hidden_size_1*dirs)
        x = self.ln1(output)
        x = self.tanh(x)
        x = self.dropout1(x)

        output, _ = self.h2(x) # -> (batch_size, seq_len, hidden_size_2*dirs)
```

```
        x = self.ln2(output)
        x = self.tanh(x)
        x = self.dropout2(x)

        output, _ = self.h3(x) # -> (batch_size, seq_len, hidden_size_3*dirs)
        x = self.ln3(output)
        x = self.tanh(x)
        x = self.dropout3(x)



        attn_output = self.attn_multi(x)  #(B, T, embed_dim=hidden_size_3*dirs)
        # Pooling over time, for example with mean pooling:
        v = attn_output.mean(dim=1)

        # Final fc layer(s)
        #x = torch.flatten(v, start_dim=1)  # Shape: (batch_size*hidden_size)
        #v = self.tanh(self.fc1(v))
        x = self.fc_out(v)
        return x
```

A.4.3 CNN-Transformer

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, dropout=0.1, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        # Term for division: 10000^(2i/d_model) -> exp(2i * -log(10000)/d_model)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() *\
                             (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term) # Even indices
        pe[:, 1::2] = torch.cos(position * div_term) # Odd indices
```

```
        pe = pe.unsqueeze(0) # Add batch dimension: (1, max_len, d_model)
        # Register as buffer: part of model's state_dict,
        # but not a parameter trained by optimiser
        self.register_buffer('pe', pe)

    def forward(self, x):
        """
        Args:
            x: Tensor, shape (batch_size, seq_len, embedding_dim)
        """
        # Add positional encoding up to the length of the input sequence
        # self.pe is (1, max_len, d_model), select slice (1, seq_len, d_model)
        # detach if PE is fixed (sinusoidal)
        x = x + self.pe[:, :x.size(1), :].detach()
        x = self.dropout(x)
        return x


class EEGNet_custom(nn.Module):
    """
    EEGNet based on the paper: "EEGNet: A Compact Convolutional Neural
                                Network for EEG-based Brain-Computer Interfaces"
    Transformer based on the paper: "Attention is all you need"

    Args:
        n_channels (int): Number of EEG channels.
        sequence_length (int): Number of time points in the sequence.
        n_classes (int): Number of output classes.
        F1 (int): Number of temporal filters (filters in the first conv layer).
        D (int): Depth multiplier for the depthwise convolution (number of
                spatial filters per temporal filter).
        F2 (int): Number of pointwise filters (usually F1 * D).
        kernel_length (int): Length of the temporal convolution kernel.
        pool_size_1 (int): Factor for the first temporal pooling.
```

```
        pool_size_2 (int): Factor for the second temporal pooling.
        separable_kernel_length (int): Length of the separable convolution kernel.
        dropout_rate (float): Dropout probability.
    """
    def __init__(self, n_channels, sequence_length, n_classes,
                 F1=64, D=8, F2=None, kernel_length=125, #half of sr=250 hz
                 pool_size_1=4, pool_size_2=8,
                 separable_kernel_length=64, #16 or 32 in the paper
                 dropout_rate=0.4):
        super(EEGNet_custom, self).__init__()

        if F2 is None:
            F2 = F1 * D # Number of pointwise filters

        self.n_channels = n_channels
        self.sequence_length = sequence_length
        self.n_classes = n_classes
        #self.activation = nn.ELU()

        # Block 1: Temporal Convolution + Depthwise Spatial Convolution
        self.block1 = nn.Sequential(
            # Input shape: (batch, 1, n_channels, sequence_length) after reshape
            # Temporal Convolution
            nn.Conv2d(1, F1, (1, kernel_length),
                      padding=(0, kernel_length // 2),
                      bias=False),
            nn.BatchNorm2d(F1),
            # Depthwise Spatial Convolution
            # Applies D filters per feature map (F1) across the channel dimension
            nn.Conv2d(F1, F1 * D, (n_channels, 1), groups=F1, bias=False),
            nn.BatchNorm2d(F1 * D),
            nn.ELU(),
            nn.AvgPool2d((1, pool_size_1)),
            nn.Dropout(dropout_rate)
```

```
    # -> (batch, F1*D, 1, sequence_length // pool_size_1)
)

# Block 2: Separable Convolution (Temporal Summary)
self.block2 = nn.Sequential(
    # Depthwise Separable Convolution
    # Depthwise part
    nn.Conv2d(F1 * D, F1 * D, (1, separable_kernel_length),
              padding=(0, separable_kernel_length // 2),
              groups=F1 * D,
              bias=False),
    # Pointwise part
    nn.Conv2d(F1 * D, F2, (1, 1), bias=False),
    nn.BatchNorm2d(F2),
    nn.ELU(),
    nn.AvgPool2d((1, pool_size_2)),
    nn.Dropout(dropout_rate)
    # -> (batch, F2, 1, sequence_length // (pool_size_1 * pool_size_2))
)



# Calculate the flattened size
final_seq_len = sequence_length // (pool_size_1 * pool_size_2)
# if no pooling after attention, use this:
#flattened_size = F2 * 1 * final_seq_len
# If pooling after attention:
flattened_size = F2
self.flattened_size = flattened_size

#Multi-Head Attention
self.attn_multi = MultiHeadAttention(embed_dim=F2, num_heads=32, dropout=0.2)
```

```
        # Transformer Encoder Block Components
        # Layer Normalization
        self.embed_dim = F2
        encoder_dropout = 0.1
        self.encoder_dropout = nn.Dropout(encoder_dropout)
        self.norm1 = nn.LayerNorm(self.embed_dim)
        self.norm2 = nn.LayerNorm(self.embed_dim)

        #Positional Encoding
        # +5 buffer
        self.pos_encoder = PositionalEncoding(d_model=self.embed_dim,
                                              dropout=0.2,
                                              max_len=final_seq_len + 5)

        # Feed-Forward Network
        ffn_hidden_dim = self.embed_dim * 4
        self.ffn = nn.Sequential(
            nn.Linear(self.embed_dim, ffn_hidden_dim),
            nn.GELU(),
            nn.Dropout(encoder_dropout), # Dropout within FFN
            nn.Linear(ffn_hidden_dim, self.embed_dim),
            # No dropout after the final linear layer in FFN, before residual
        )

        self.classifier = nn.Linear(flattened_size, n_classes)

    def forward(self, x):
        """
        Args:
            x (Tensor): Input EEG data tensor with shape
                        (batch_size, sequence_length, n_channels).
        Returns:
            Tensor: Output tensor with shape (batch_size, n_classes).
        """
```

```
# Reshape input to (batch_size, 1, n_channels, sequence_length)
x = x.permute(0, 2, 1) # -> (batch_size, n_channels, sequence_length)
x = x.unsqueeze(1)     # -> (batch_size, 1, n_channels, sequence_length)

# EEGNet blocks
x = self.block1(x)
x = self.block2(x)

# shape = (batch, F2=channels, 1, reduced_seq_len)
x = x.squeeze(2)
x = x.permute(0, 2, 1) # -> (batch, T_reduced, embed_dim=F2)

# Transformer Encoder Block
# Positional Encoding: PE
x = self.pos_encoder(x)
# Multi-Head Self-Attention + Residual + Norm
attn_input = x
attn_output = self.attn_multi(attn_input)
# Apply dropout before the residual connection
x = attn_input + self.encoder_dropout(attn_output)
x = self.norm1(x)
# Feed-Forward Network + Residual + Norm
ffn_input = x
ffn_output = self.ffn(x)
# Apply dropout before the residual connection
x = ffn_input + self.encoder_dropout(ffn_output)
x = self.norm2(x)
# Mean Pooling
x = x.mean(dim=1)      # -> (B, F2=embed_dim)

# Flatten for classifier, if needed
#x = x.reshape(x.size(0), -1) # Flatten all dimensions except batch
#x = torch.flatten(x, start_dim=1)
```

```
#Final classifier
x = self.classifier(x)
return x
```